

Módulo 4. *React* avanzado. *Testing* de aplicaciones web

Introducción

En este módulo de estudio, exploramos distintos aspectos clave de desarrollo web, comenzando por abordar los conceptos de eventos y manipulación del DOM para lograr interacciones dinámicas en nuestras aplicaciones. Luego, nos adentraremos en la estilización y renderizado de componentes utilizando CSS y ReactDOM. Continuaremos explorando el ruteo y el consumo de API, lo que nos permitirá crear aplicaciones con múltiples vistas y obtener datos externos.

Además, introduciremos a React Native, un *framework* que nos brinda la capacidad de desarrollar aplicaciones móviles nativas y extiende las capacidades base que ofrece React. Vamos, entonces, a profundizar en estos nuevos temas de estudio.

Por último, repasaremos los fundamentos enfocados en el universo del *testing* de aplicaciones *web frontend*, viendo las principales herramientas del mercado orientadas al *testing* manual y automatizado, además de conocer otras tantas opciones modernas, disponibles como parte de las herramientas para desarrollador integradas en los principales navegadores web.

Video de inmersión

Unidad 1: React avanzado. React Native

La librería React cuenta con muchas funcionalidades que potencian toda la interacción del usuario con los diferentes elementos gráficos de una aplicación web. Entre estas, encontramos el manejo de DOM, trabajo con estilos CSS, todo controlado desde esta librería basada en componentes, y también el escenario de detección y reacción a los

eventos generados por un usuario.

Además, las bases de la librería React son los fundamentos básicos necesarios para luego poder adentrarnos en React Native, una herramienta que potencia la lógica de React introduciendo la posibilidad de crear aplicaciones nativas *desktop* y *mobile*, a partir de las bases del desarrollo web *frontend*.

Tema 1: Manejo de eventos con React JS

El manejo de eventos con React JS es una parte esencial para crear aplicaciones interactivas y dinámicas. React JS nos proporciona una forma sencilla y eficiente de manejar eventos, lo que nos permite a nosotros, desarrolladores, crear interfaces de usuario altamente receptivas.

La forma de manejar eventos con esta librería es mediante las estructuras de archivos JSX, en donde asignamos una función de controlador de eventos a un elemento específico.

Para entender este escenario, proyectemos lo que oportunamente tratamos en las unidades de estudio dedicadas a JavaScript, orientadas al manejo de eventos, como un ejemplo válido para aplicar.

Figura 1. Código



```
Eventos

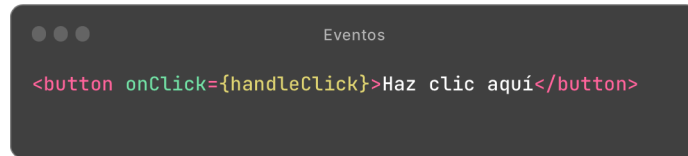
// HTML
<button id="btnEnviar">ENVIAR</button>

// JavaScript
const btnEnviar = document.querySelector('button#btnEnviar')
btnEnviar.addEventListener('click', () => console.log('maneja el click'))
```

Fuente: elaboración propia.

Está misma estructura, al momento de ser llevada a React, se simplifica a través de un modelo similar al siguiente.

Figura 2. Código



```
Eventos

<button onClick={handleClick}>Haz clic aqui</button>
```

Fuente: elaboración propia.

En el bloque de código del ejemplo anterior, vemos que el nombre del evento que se va a controlar se define dentro del componente (**button**, en este caso), tal como cuando referenciábamos antiguamente el atributo **onclick** dentro de un elemento HTML.

Y la función **handleClick** será invocada cada vez que el usuario haga clic en el botón en cuestión. Esta función permite realizar cualquier acción necesaria que se nos ocurra, como ser: actualizar el estado de una aplicación, llamar una API REST, etcétera.

Tipos de eventos en React

Además del evento de **click**, React JS nos proporciona un manejo de eventos para una amplia gama de interacciones, como eventos de cambio, eventos de teclado y eventos de formulario. Prácticamente todos los eventos que vimos con JavaScript están disponibles también en React. Algunos ejemplos son los siguientes.

Figura 3. Eventos en React



```
Eventos React

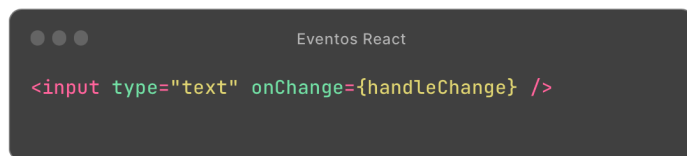
<form onSubmit={handleSubmit}>
  /* Contenido del formulario {various input types} */
  <button type="submit">Enviar</button>
</form>
```

Fuente: elaboración propia.

Imaginemos un formulario HTML donde tenemos un componente llamado **form**, y dentro de este, diferentes *input types*. Cuando el usuario pulse el botón *Enviar*, característico

de un formulario web, este disparará el evento **onSubmit**, que tendrá la función **handleSubmit**, la cual se ocupará de manejar el contenido cargado en el formulario HTML.

Figura 4. Código



```
<input type="text" onChange={handleChange} />
```

Fuente: elaboración propia.

Dentro del formulario HTML, tendremos varios *input types*. Cada uno de ellos puede detectar la interacción del usuario cuando escribe contenido en los campos. Así, cada uno de sus campos podrá manejar alguna interacción por parte del usuario con el evento **onChange**, y este invocará a una función llamada **handleChange** que nos permite realizar alguna tarea asociada al evento en cuestión.

Como podemos apreciar, el manejo de eventos con React JS nos permite crear aplicaciones altamente interactivas y receptivas. Al asignar funciones de controlador de eventos a los elementos de la interfaz de usuario, podemos definir cómo reaccionará la aplicación ante diferentes acciones del usuario. Esto le proporciona a este último una experiencia fluida y agradable.

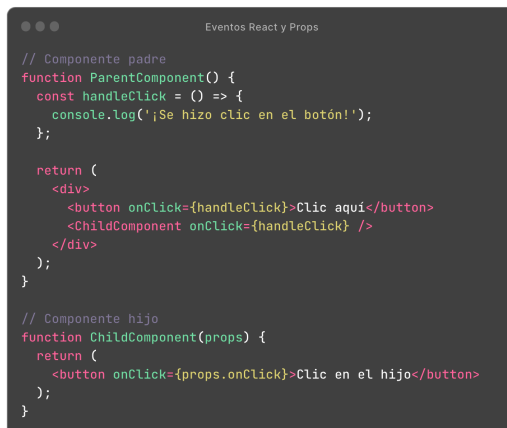
Interacción con *props* y estados

En React JS, los eventos y las *props* interactúan sin problema para permitir la comunicación entre los componentes. Cuando definimos eventos en alguno de los componentes de React, debemos asignarles una función para una determinada acción o evento, como, por ejemplo: hacer clic en un botón.

Debemos pasar esta función como una *prop* al componente hijo, quien será el que necesite manejar ese evento en cuestión. Para entender esto, imaginemos tener un componente padre que posee un botón, además de contener un componente hijo.

El componente padre definirá una función llamada **handleClick**, la cual se ejecutará cuando se haga clic en el botón. Luego, el componente padre pasa esta función como una *prop* llamada **onClick** al componente hijo.

Figura 5. Código



```

// Componente padre
function ParentComponent() {
  const handleClick = () => {
    console.log('Se hizo clic en el botón!');
  };

  return (
    <div>
      <button onClick={handleClick}>Clic aqui</button>
      <ChildComponent onClick={handleClick} />
    </div>
  );
}

// Componente hijo
function ChildComponent(props) {
  return (
    <button onClick={props.onClick}>Clic en el hijo</button>
  );
}

```

Fuente: elaboración propia.

En este último ejemplo, vemos que tanto el botón en el componente padre como el botón en el componente hijo tienen asignados el mismo evento **onClick**, el cual hace referencia a la *prop* que contiene la función **handleClick**. Entonces, cuando se haga clic en cualquiera de los botones, se ejecutará la función **handleClick** definida en el componente padre, y se mostrará el mensaje definido en la consola.

De esta forma es como los eventos en React se comunican mediante el paso de funciones como *props* a los componentes, lo cual permite una interacción y manejo de eventos en diferentes partes de la aplicación.

Recordemos que las *props* son habitualmente utilizadas para pasar datos de un componente padre a un componente hijo. Estas propiedades son inmutables y se utilizan para transmitir información y configuración a través de la jerarquía de componentes.

Cuando pasamos *props*, los componentes reciben datos específicos y realizan acciones

en función de ellos.

Por otro lado, habíamos visto que el estado (*state*) es, básicamente, un objeto que contiene datos modificables internamente en un componente. A diferencia de las *props*, el estado es administrado y controlado internamente por el componente.

Entonces, cuando el estado cambia, es React JS quien se encarga automáticamente de actualizar y renderizar nuevamente aquellos componentes afectados.

La interacción entre las *props* y el estado ocurre cuando las *props* se utilizan para inicializar el estado de un componente o cuando el estado se actualiza en función de las *props*. Esto permite que los componentes React sean más dinámicos y adaptables a diferentes situaciones.

Por ejemplo, supongamos que tenemos un componente llamado *Contador* que muestra un número y tiene dos botones: uno para incrementar el número y otro para decrementarlo. Allí podríamos utilizar una *prop* denominada **valorInicial** para pasar el número inicial al componente; luego, utilizamos el estado para realizar las operaciones referidas al incremento y decremento de su valor.

Figura 6. Código

```
class Contador extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      valor: props.valorInicial,
    };
  }

  incrementar = () => {
    this.setState((prevState) => ({ valor: prevState.valor + 1, }));
  };

  decrementar = () => {
    this.setState((prevState) => ({ valor: prevState.valor - 1, }));
  };

  render() {
    return (
      <div>
        <p>Valor: {this.state.valor}</p>
        <button onClick={this.incrementar}>Incrementar</button>
        <button onClick={this.decrementar}>Decrementar</button>
      </div>
    );
  }
}
```

Fuente: elaboración propia.

En este último ejemplo, el valor inicial es enviado al componente a través de la *prop* **valorInicial**. Luego, ese valor es utilizado para inicializar el estado del componente en el constructor. Los métodos **incrementar()** y **decrementar()** modifican el estado y actualizan el valor del contador.

Tal como vemos en este último ejemplo funcional, la interacción entre *props* y estado nos permite crear componentes flexibles y personalizables. Las *props* en sí proporcionan la capacidad de configurar un componente desde el exterior, mientras que el estado nos permite que el componente realice cambios internos y responda a las interacciones del usuario en general.

Renderizado condicional

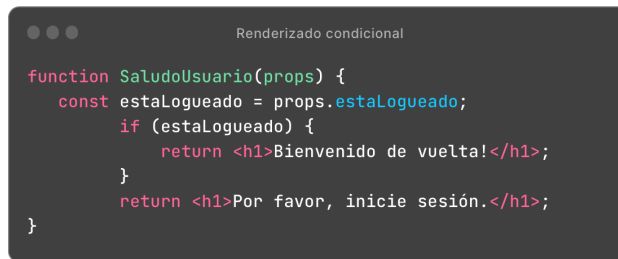
En React JS, encontraremos de forma frecuente la necesidad de renderizar diferentes componentes o de mostrar diferentes comportamientos en función de ciertas condiciones. Este concepto es conocido como renderizado condicional.

El renderizado condicional en React se comporta en la misma manera que las condiciones funcionan en JavaScript (condiciones = if u operador ternario).

En el lenguaje JavaScript, utilizamos habitualmente la cláusula **if** o el operador ternario (**? :**). Esto mismo es totalmente trasladable a React cuando necesitamos crear elementos representando el estado actual, para así permitir que React sea quien actualice la interfaz de usuario para que coincida con ellos.

Para comprender mejor esto, veamos, a continuación, un ejemplo de renderizado condicional, aplicado a la librería React JS. Imaginemos que contamos con un componente que muestra el mensaje de bienvenida al usuario, si es que este se encuentra identificado (*logueado*) en la aplicación web.

Figura 7. Código



```

Renderizado condicional

function SaludoUsuario(props) {
  const estaLogueado = props.estaLogueado;
  if (estaLogueado) {
    return <h1>Bienvenido de vuelta!</h1>;
  }
  return <h1>Por favor, inicie sesión.</h1>;
}

```

Fuente: elaboración propia.

En este último ejemplo, podemos ver que la variable **estaLogueado** será la que determine qué mensaje se renderizará en pantalla.

Veamos, a continuación, otro ejemplo de uso de renderizado condicional. Esta vez, utilizando el **operador ternario** en lugar de la cláusula **if**.

Figura 8. Código



```

Renderizado condicional

function BotonSesion(props) {
  const estaLogueado = props.estaLogueado;
  return (
    <button onClick={props.clickHandler}>
      {estaLogueado ? 'Cerrar sesión' : 'Iniciar sesión'}
    </button>
  );
}

```

Fuente: elaboración propia.

En este último ejemplo, el componente **button** cambiará el texto que se va a mostrar entre *Iniciar sesión* y *Cerrar sesión*, dependiendo del estado de la constante **estaLogueado**. El operador ternario es aprovechado a través de su funcionalidad nativa de retorno implícito, para devolver uno u otro texto, de acuerdo al estado de la constante definida.

Componentes que previenen su propio renderizado

Ante determinadas situaciones, un componente React tiene la capacidad de prevenir su propio renderizado. Para lograr esta funcionalidad, debemos retornar **null** desde su función de renderizado. Veamos cómo realizarlo a través del siguiente ejemplo.

Figura 9. Código



```
function Advertencia(props) {  
  if (!props.warn) {  
    return null;  
  }  
  return (  
    <div className="warning">  
      Advertencia!  
    </div>  
  );  
}
```

Fuente: elaboración propia.

En este último ejemplo de código, el componente **Advertencia** se renderizará solo si la *prop* **warn** posee un valor booleano del tipo **true**. Si la *prop* posee un valor booleano del tipo **false**, en ese caso, el componente no se renderizará.

Cómo mostrar u ocultar componentes y elementos en una página

Otra de las características que hace a React tan útil es su capacidad nata de renderizar condicionalmente componentes y elementos, lo cual permite mostrar u ocultar determinadas partes en la interfaz de usuario, en función de ciertas condiciones.

Un enfoque común para mostrar u ocultar un componente es utilizar un estado propio de esta librería para controlar su visibilidad. Para representar esto, podemos imaginar que tenemos un componente llamado **Mensaje**, el cual queremos mostrar u ocultar cuando se hace clic en un determinado botón.

Figura 10. Código

```

Renderizado condicional

const [mostrarMensaje, setMostrarMensaje] = useState(false);

function App() {
  const [mostrarMensaje, setMostrarMensaje] = useState(false);
  return (
    <div>
      <button onClick={() => setMostrarMensaje(!mostrarMensaje)}>
        {mostrarMensaje ? 'Ocultar' : 'Mostrar'} Mensaje
      </button>
      {mostrarMensaje && <Mensaje />}
    </div>
  );
}

function Mensaje() {
  return <h1>Hola, mundo!</h1>;
}

export default App;

```

Fuente: elaboración propia.

En este ejemplo, usamos el **hook useState** para crear un estado llamado **mostrarMensaje**, el cual controla si el componente **Mensaje** se debe mostrar o no. Cuando hacemos clic en el botón, llamamos a **setMostrarMensaje** con el valor opuesto al estado actual de **mostrarMensaje**, efectivamente, cambiando entre **true** (mostrar) y **false** (ocultar).

Seguramente, en determinados escenarios tengamos la necesidad de no querer mostrar u ocultar un componente completo, sino solo ciertos elementos dentro de este componente. Esto también es totalmente realizable con el uso de renderizado condicional.

Veamos a continuación cómo lograrlo.

Figura 11. Código

```

Renderizado Condicional

import React, { useState } from "react";

function App() {
  const [mostrarDetalle, setMostrarDetalle] = useState(false);
  return (
    <div>
      <h1>Producto:</h1>
      <p>Descripción breve del producto.</p>
      {mostrarDetalle && <p>Esta es una descripción detallada del producto.</p>}
      <button onClick={() => setMostrarDetalle(!mostrarDetalle)}>
        {mostrarDetalle ? 'Ocultar Detalles' : 'Mostrar Detalles'}
      </button>
    </div>
  );
}

export default App;

```

Fuente: elaboración propia.

En este último ejemplo, podemos ver un componente denominado **App**, el cual muestra información sobre un producto específico. Al hacer clic en el botón, se muestra u oculta

una descripción detallada del producto, combinando en su visualización u ocultamiento el uso de un condicional que se ocupa de analizar el estado de visualización actual.

Como podemos apreciar hasta aquí, React JS nos facilita la creación de interfaces de usuario dinámicas y personalizadas aplicando metodologías de uso de renderizado condicional y de manejo de estados.

Ya sea que estemos mostrando u ocultando componentes completos o elementos individuales dentro de un componente, React nos ofrece siempre las herramientas necesarias para lograr nuestro cometido de una manera simple y efectiva.

Actividades de repaso

Cuando utilizamos renderizado condicional de componentes, la opción más efectiva para controlar esto con React es:

Manejando el renderizado con un condicional if – else.

Manejando el renderizado con un operador ternario.

Ambas opciones anteriores son válidas.

Al utilizar state en componentes React, este se diferencia del uso de props porque:

State es un objeto que puede modificar sus valores internamente en un componente.

Prop es un objeto que permite modificar sus valores internamente en un componente mientras que state no lo permite.

Ambos funcionan de manera similar, permiten que sus valores se modifiquen internamente en un componente.

Al necesitar controlar eventos desde un componente React, encontramos que dichos eventos se manejan:

A través de nombres de eventos propios de React.

Referenciando el nombre del evento de manera similar a los atributos que se utilizaban en HTML.

Intercediendo con el método `addEventListener` de JavaScript.

Tema 2: CSS y ReactDOM

Repasando un poco los conocimientos de estilos que se aplican sobre proyectos web *frontend*, nos referimos actualmente a CSS como un lenguaje de estilo que nos permite definir la apariencia y el diseño de los elementos HTML en una página web, controlar aspectos tales como: el color, tamaño, tipografía y la disposición de los elementos en un documento HTML.

Y cuando CSS entra en el contexto de React, el manejo gráfico que nos da este lenguaje juega un papel fundamental en la aplicación de estilos a los diferentes componentes y elementos que creamos con esta librería JS.

Aplicar estilos en componentes y elementos

Para entender cómo conjugar correctamente el manejo de estilos con el uso de React JS, debemos introducir a ReactDOM como el camino más apropiado para un exitoso manejo de estilos. Por lo tanto, podemos definir a ReactDOM como una librería que complementa a React, utilizada para renderizar los componentes en el DOM de una página web.

ReactDOM nos brinda la vía apropiada para integrar a React con el HTML existente y facilita todo lo referente a la manipulación de elementos en el DOM.

Cuando tratamos de aplicar estilos en componentes y/o elementos React, podemos utilizar diferentes enfoques. Uno de los métodos más comunes es utilizando el CSS tradicional. Esto nos permite crear archivos CSS separados y luego importarlos en cada componente que creamos dentro de React. Así, lograremos definir reglas de estilo específicas para cada componente y aplicarlas de manera efectiva mediante la asignación de clases o identificadores a los elementos en cuestión.

Imaginemos que tenemos un componente **button React** y debemos aplicarle un estilo CSS personalizado. Para ello, debemos crear una clase CSS en el archivo homónimo, agregando los estilos necesarios. Luego, asignaremos la clase CSS al elemento **button** dentro del componente apropiado, utilizando para ello la propiedad **className**.

Figuras 12 y 13. Código

```
/* styles.css */
.customButton {
  background-color: #f2f2f2;
  color: #333;
  padding: 10px 20px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}
```

```
import React from 'react';
import './styles.css'; // Importamos el archivo CSS

function CustomButton() {
  return (
    <button className="customButton">
      Haz clic aquí
    </button>
  );
}

export default CustomButton;
```

Fuente: elaboración propia.

Así, logramos aplicar nuevamente la clase **import** nativa de JS, para importar dentro de nuestro archivo JSX la hoja de estilo apropiada y obtener de ella la clase CSS correspondiente, que terminaremos referenciando sobre el *tag* **<button>**. Recordemos aquí que estamos trabajando con JS y que el atributo **class** que utilizan los elementos HTML convencionales se debe trabajar como una propiedad denominada **className**, del lado de JavaScript.

Estilos en línea

Además del enfoque tradicional de CSS, React también cuenta con la posibilidad de aplicar estilos en línea utilizando JavaScript. Para ello, podemos definir los estilos directamente en los elementos de los componentes React, utilizando, en este caso, la propiedad **style**.

Esta opción es muy útil cuando necesitas aplicar estilos dinámicos basados en el estado (**state**) o las propiedades (*props*) del componente React en sí.

Para entender esto, analicemos el siguiente ejemplo de código y concentrémonos específicamente en el objeto llamado **buttonStyles**.

Figura 14. Código

```
import React, { useState } from 'react';

function MyComponent() {
  const [isHighlighted, setIsHighlighted] = useState(false);

  const handleMouseOver = () => {
    setIsHighlighted(true);
  };

  const handleMouseOut = () => {
    setIsHighlighted(false);
  };

  const buttonStyles = {
    backgroundColor: isHighlighted ? 'blue' : 'gray',
    color: 'white',
    padding: '10px 20px',
    borderRadius: '5px',
    cursor: 'pointer',
  };

  return (
    <button
      style={buttonStyles}
      onMouseOver={handleMouseOver}
      onMouseOut={handleMouseOut}
    >
      Haz clic aqui
    </button>
  );
}

export default MyComponent;
```

Fuente: elaboración propia.

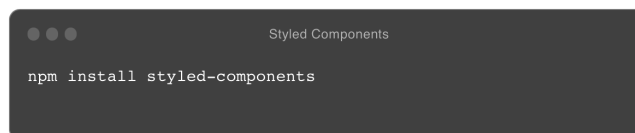
Como podemos apreciar en este último ejemplo, el componente **MyComponent** renderiza un elemento de botón que utiliza la propiedad **style** para aplicar los estilos en línea necesarios. Estos estilos se definen en el objeto **buttonStyles** dentro del componente. Además, la propiedad **backgroundColor** cambia dinámicamente entre **blue** y **gray**, basado en el estado de **isHighlighted**. Cuando el cursor se coloca sobre el botón, el estado **isHighlighted** se actualiza para cambiar el color de fondo.

Styled Components

Styled Components es otra librería popular para utilizar en el ecosistema de React, la cual permite definir estilos en línea de una manera intuitiva y poderosa. En lugar de definir archivos CSS por separado, Styled Components nos permite escribir estilos directamente en nuestros componentes React, utilizando una sintaxis similar a CSS, pero con el poder y la flexibilidad de JavaScript.

Si deseamos utilizar Styled Components, debemos instalarlo en nuestro proyecto web basado en React. Para ello, ejecutamos el siguiente comando en la ventana Terminal.

Figura 15. Código

A terminal window with a dark background and light gray text. The title bar at the top reads "Styled Components". The command "npm install styled-components" is entered in the terminal.

```
npm install styled-components
```

Fuente: elaboración propia.

Con Styled Components instalado, debemos importarlo en todos los archivos de componentes React donde deseemos utilizarlo. Veamos un ejemplo de cómo implementarlo.

Figura 16. Código



```
import React from 'react';
import styled from 'styled-components';

const styledButton = styled.button`
  background-color: #f2f2f2;
  color: #333;
  padding: 10px 20px;
  border: none;
  border-radius: 4px;
  font-size: 16px;
  cursor: pointer;

  &:hover {
    background-color: #ddd;
  }
`;

function MyComponent() {
  return (
    <styledButton>Haz clic aquí</styledButton>
  );
}

export default MyComponent;
```

Fuente: elaboración propia.

En el ejemplo anterior, creamos un componente de botón estilizado utilizando Styled Components. A través de la función **styled**, definimos un componente llamado **StyledButton** que representa un botón. En el bloque *template string* dentro del componente **styled.button**, aplicamos las reglas de estilo utilizando la sintaxis nativa de CSS.

A continuación, en el componente **MyComponent**, simplemente, usamos **<StyledButton>** en lugar de **<button>** para renderizar nuestro botón estilizado. De esta manera, todos los estilos definidos en el componente **StyledButton** se aplicarán automáticamente al elemento **<button>**.

Una de las ventajas clave de Styled Components es su capacidad para aceptar propiedades (*props*) y estilos dinámicos. Por lo tanto, podemos pasar *props* a los componentes estilizados y utilizarlas para cambiar los estilos en función de las necesidades.

Veamos el siguiente ejemplo.

Figura 17. Código

```
Styled Components

const StyledButton = styled.button`
  background-color: ${props => props.primary ? 'blue' : 'gray'};
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 4px;
  font-size: 16px;
  cursor: pointer;
  &:hover {
    background-color: ${props => props.primary ? 'darkblue' : 'darkgray'};
  };
};

const MyComponent = () => {
  return (
    <div>
      <h1>Styled Components en React</h1>
      <StyledButton primary>Primary Button</StyledButton>
      <StyledButton>Secondary Button</StyledButton>
    </div>
  );
};
```

Fuente: elaboración propia.

En este nuevo ejemplo, integramos una *prop* **primary** dentro del componente **StyledButton**. Dependiendo del valor de **primary**, aplicaremos diferentes estilos sobre el botón. Así logramos crear botones con estilos diferentes, según el propósito de estos.

Como podemos apreciar, seguimos integrando el uso de *Template Strings*, ahora con *Template Literals* en combinación con el operador ternario, quien aplicará el valor indicado a las propiedades CSS correspondientes.

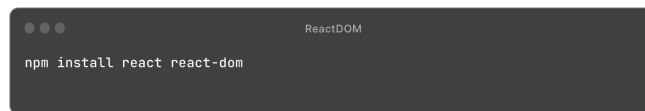
Styled Components también admite la herencia de estilos. Esto nos habilita a crear componentes estilizados, basados en otros componentes estilizados existentes y así lograr extender sus estilos.

Renderizar componentes React utilizando ReactDOM

Uno de los conceptos fundamentales de React es la capacidad de crear y renderizar componentes reutilizables en una aplicación web. Una vez que hayamos definido los componentes React necesarios, debemos renderizarlos en el DOM para que estos sean visibles en la aplicación web ya cargada en el browser. A esto será posible realizarlo utilizando la librería ReactDOM.

Como siempre, el primer paso será instalarla dentro de nuestro proyecto React.

Figura 18. Código



```
npm install react react-dom
```

Fuente: elaboración propia.

Luego de este proceso, debemos importar **ReactDOM** en el archivo principal de nuestra aplicación: **index.js** o **app.js**, dependiendo el nombre que hayamos elegido para este.

Ya importada la librería ReactDOM, podemos utilizar el método **ReactDOM.render()** para renderizar los componentes React en el elemento del DOM especificado. Teniendo un componente llamado **App**, el cual es el componente principal de la aplicación, debemos renderizar de la siguiente manera.

Figura 19. Código



```
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App';

ReactDOM.render(<App />, document.getElementById('root'));
```

Fuente: elaboración propia.

El método **ReactDOM.render()** acepta dos argumentos específicos:

1. el componente de React que deseas renderizar;
2. el elemento del DOM donde deseas renderizarlo.

Implementamos, finalmente, **document.getElementById('root')** para obtener el elemento del DOM con el ID especificado como **root**.

Actualizar componentes renderizados

Con el componente principal ya renderizado, React se ocupará de actualizar automáticamente estos componentes renderizados, cuando sea necesario. Esto se debe

a la naturaleza declarativa de React, donde definimos solamente cómo debería verse nuestra UI, en función del estado actual; luego, React se encarga de mantenerla actualizada.

En el momento en el cual actualicemos el estado o propiedades de un componente, React comparará este nuevo estado o las propiedades en cuestión con el estado o propiedades anteriores, y así determinará los cambios necesarios que se deben aplicar en el DOM.

Solo se actualizarán aquellos elementos del DOM que hayan cambiado. Esto mejora el rendimiento y la eficiencia de toda aplicación construida con React. Para entender esto, podemos tomar como referencia el anterior ejemplo de código, donde tenemos un componente llamado **Counter**, que muestra un contador que incrementa o decrementa un valor numérico.

En este componente, cada vez que el usuario haga clic en el botón en cuestión, el estado del contador se actualiza y React solo se ocupará de actualizar el componente **Counter** dentro de la interfaz DOM HTML.

Actividades de repaso

Cuando hablamos de estilos en línea dentro de los componentes React, nos referimos:

A la definición de estilos CSS agregando el atributo style en el componente.

A la definición de estilos CSS utilizando clases CSS en el componente.

El documento HTML es quien aplica los estilos en los componentes React.

La correcta definición de Styled Components es:

Un nombre genérico para los estilos en línea y basados en clases en React.

Una librería React que permite definir estilos en los documentos HTML que representan los componentes React.

Una librería utilizada en React para definir estilos en línea de manera intuitiva.

Cuando aplicamos styled components en React, debemos definir los estilos utilizando:

Solo props primary en los componentes HTML definidos como elementos.

Definiéndolos mediante Template String y Template Literals.

Recurriendo a la propiedad styles, nativa de JavaScript.

Tema 3: Ruteo y API

Otra de las fabulosas características que encontramos en React JS es la capacidad de crear rutas dinámicas en cualquier aplicación web. En el cierre del módulo anterior, hablamos de las características principales de una aplicación SPA basada en JavaScript nativo. Si bien existe la posibilidad de manejar esto de una manera fácil y de otra forma no tan simple con JS nativo, React aporta un valor agregado en este terreno, ya que facilita la implementación de aplicaciones SPA basadas en ruteo mediante URL.

Veamos, entonces, de qué se trata esto y cómo React se ocupa del manejo de rutas de forma efectiva, para lograr aplicaciones SPA de calidad.

React Router

Para gestionar las rutas en una aplicación React, podemos hacer uso de una librería llamada React Router. Esta nos brinda un conjunto de componentes y utilidades que facilitan la creación de rutas dinámicas en nuestras aplicaciones web.

Para comenzar a utilizarla, primero debemos instalarla a través de la ventana *Terminal*.

Figura 20. Código



```
React Router  
npm install react-router-dom
```

Fuente: elaboración propia.

Luego de instalada, debemos importar los componentes necesarios y, seguido a esto, configurar nuestras rutas.

En su forma más simple, podemos utilizar el componente **BrowserRouter** para envolver nuestra aplicación y definir todas las rutas necesarias para esta. Dentro del componente **BrowserRouter**, podemos utilizar el componente **Route**, el cual nos permite asociar una ruta específica con un componente React determinado.

Veamos un ejemplo de código con dos rutas diferentes: la página principal "/" y el apartado nosotros, definido como "/about".

Figura 21. Código



```
React Router  
  
import React from 'react';  
import { BrowserRouter, Route, Link } from 'react-router-dom';  
const Home = () => {  
  return <h1>¡Página principal!</h1>;  
}  
  
const About = () => {  
  return <h1>¡Acerca de nosotros!</h1>;  
}  
  
const App = () => {  
  return (  
    <BrowserRouter>  
      <nav>  
        <ul>  
          <li>  
            <Link to="/">Inicio</Link>  
          </li>  
          <li>  
            <Link to="/about">Nosotros</Link>  
          </li>  
        </ul>  
      </nav>  
      <Route path="/" exact component={Home} />  
      <Route path="/about" component={About} />  
    </BrowserRouter>  
  );  
}  
export default App;
```

Fuente: elaboración propia.

En el ejemplo anterior, tenemos creados dos componentes: **Home** y **About**. Integramos el componente **BrowserRouter** para envolver nuestra aplicación y luego el componente **Route**, que nos permite definir las rutas y asociarlas con los componentes

correspondientes.

Dentro del componente **App**, agregamos un elemento de navegación denominado **nav**, el cual contiene enlaces **Link** a nuestras rutas. Cuando hacemos clic en uno de los enlaces, React Router se ocupará de actualizar la URL y mostrar el componente asociado a esa ruta.

Es importante destacar que podemos utilizar diferentes propiedades en el componente **Route** para controlar el comportamiento de las rutas. Dentro de estas propiedades, encontramos a **exact**, una propiedad utilizada para indicar una coincidencia exacta de la ruta, además de la propiedad **component**, para especificar el componente que se desea renderizar.

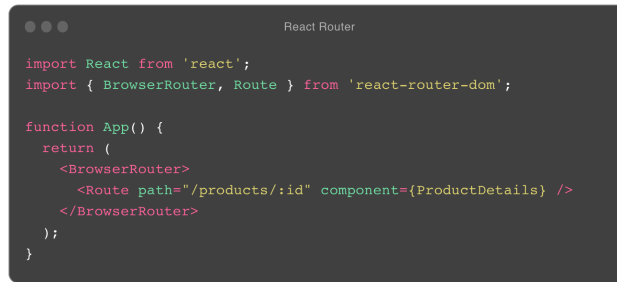
Además de **BrowserRouter** y **Route**, React Router nos brinda otros componentes y utilidades avanzadas para la gestión de rutas anidadas, también para pasar parámetros a través de las rutas y para realizar redireccionamientos, entre otras funcionalidades. Esto nos aporta una gran flexibilidad para crear aplicaciones con una navegación compleja y dinámica.

Navegación utilizando parámetros dinámicos

Los parámetros dinámicos en las rutas de navegación nos permiten capturar valores variables en la URL, lo que nos da flexibilidad al manejar diferentes casos y datos específicos. Si queremos utilizar parámetros dinámicos en React Router, debemos definir la ruta con una parte variable en forma de marcador de posición utilizando la notación `:` seguida del nombre del parámetro en cuestión.

Por ejemplo, imaginemos que tenemos una ruta dedicada a mostrar detalles de un producto, donde el ID del producto es variable. Podemos definir esto de la siguiente manera.

Figura 22. Código



```

React Router

import React from 'react';
import { BrowserRouter, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Route path="/products/:id" component={ProductDetails} />
    </BrowserRouter>
  );
}

```

Fuente: elaboración propia.

En el ejemplo de código anterior, definimos la ruta **/products/:id** donde **:id** corresponde al parámetro dinámico que representa el ID del producto. Cuando se acceda a una URL como **/products/123**, React Router capturará el valor 123 en cuestión y lo pasará como una **prop** llamada **match**, al componente **ProductDetails**.

Luego, para acceder a los parámetros dinámicos en el componente **ProductDetails**, podemos utilizar **props.match.params.id** para obtener el valor del ID del producto.

Figura 23. Código



```

React Router

import React from 'react';

function ProductDetails(props) {
  const productId = props.match.params.id;

  // Lógica adicional usando el ID del producto

  return (
    <div>
      <h2>Detalles del producto {productId}</h2>
      {/* Resto del contenido del componente */}
    </div>
  );
}

```

Fuente: elaboración propia.

En este último ejemplo de código, accedemos al parámetro dinámico **id** utilizando **props.match.params.id** en el componente **ProductDetails**. Luego, utilizamos ese valor en la lógica de nuestra aplicación, para mostrar los detalles correspondientes del producto con ese ID. Si tenemos conocimientos de los aspectos básicos de *endpoints* en API Restful, el mecanismo que propone React Router es similar al del escenario *backend*.

API Restful con React

En el desarrollo de aplicaciones web con React, es igual de común que con JavaScript nativo interactuar con servicios externos obteniendo datos de una API o enviando información a través de peticiones HTTP. Para lograr esto, React nos ofrece diversas herramientas y enfoques para integrar y consumir API de manera eficiente.

En React, podemos utilizar alguna librería externa para lograr este cometido, como lo es Axios, o también recurrir a su forma nativa, utilizando la función **Fetch**.

Axios, a diferencia de Fetch, es una librería de terceros que ofrece una interfaz sencilla y amigable para realizar peticiones HTTP, con características adicionales, como manejo de errores y configuraciones personalizadas.

Veamos, a continuación, un ejemplo sobre cómo realizar una solicitud GET a una API utilizando la Fetch API en una aplicación React.

Figura 24. Código



```
API Restful con React

import React, { useEffect, useState } from 'react';
const MyComponent = () => {
  const [data, setData] = useState([]);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(response => response.json())
      .then(data => setData(data));
  }, []);

  return (
    <div>
      {data.map(item => (
        <p key={item.id}>{item.title}</p>
      ))}
    </div>
  );
}

export default MyComponent;
```

Fuente: elaboración propia.

En este ejemplo de código, utilizamos el **hook useEffect** para realizar una solicitud **GET** a una API cuando el componente se monta. La respuesta de la API se convierte en formato JSON utilizando el método **.json()** y luego se establece en el estado del componente utilizando **setData**. Finalmente, se muestra la información obtenida de la API en la interfaz de usuario.

No podemos dejar de mencionar que también es posible realizar solicitudes **POST**, **PUT**, **DELETE** u otras operaciones, tanto con Fetch como también con Axios. Estas solicitudes permiten enviar datos a la API, actualizar recursos y eliminar datos, entre otras acciones.

Actividades de repaso

Cuando pensamos en incluir módulos JS en una aplicación, una de las cuestiones que debemos tener en cuenta para crear funciones reutilizables es que estas:

Estén declaradas de la forma más reutilizable posible.

Cuenten con el atributo `type="module"` definido en la función.

Utilicen la cláusula `import` como parámetro entre sus paréntesis.

Identifica dos características que nos benefician cuando importamos funciones desde un archivo JS a otro archivo JS:

Duplicar funciones con la misma lógica.

Declarar archivos JS al final de todos los documentos HTML.

Cargar en memoria código JS innecesario.

Cuando declaramos una función anteponiendo la palabra reservada `export`, logramos:

Convertir a la función en un objeto importable.

Convertir a la función en un módulo JavaScript.

Convertir a la función en un objeto exportable.

Con el auge de la tecnología móvil y la demanda creciente de aplicaciones para dispositivos móviles, React Native ha surgido como una poderosa herramienta para el desarrollo de aplicaciones móviles multiplataforma. Al utilizar JavaScript como lenguaje principal, React Native nos brinda la capacidad de crear aplicaciones nativas para iOS y Android con un enfoque eficiente y unificado.

A continuación, exploraremos los fundamentos de React Native y descubriremos cómo esta tecnología revolucionaria ha simplificado el proceso de desarrollo móvil y ha permitido a los desarrolladores crear aplicaciones de alta calidad y rendimiento para millones de usuarios en todo el mundo.

Fundamentos

React JS y React Native son dos tecnologías relacionadas, pero con enfoques y usos diferentes. Ambas están basadas en la biblioteca React de JavaScript y son desarrolladas y mantenidas por *Meta Inc.* (Facebook).

Diferencias entre tecnologías

Anteriormente, vimos que React JS es una librería de JavaScript utilizada para desarrollar interfaces de usuario interactivas en aplicaciones web. Permite crear componentes reutilizables, los cuales se encargan de renderizar y actualizar la interfaz de usuario de manera eficiente. Todo el trabajo que realiza React JS lo hace utilizando el DOM del navegador para representar la interfaz de usuario y se ejecuta en un entorno de navegador.

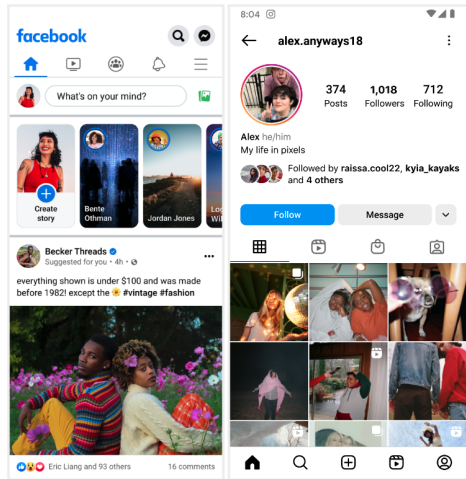
Por otro lado, React Native es un marco de desarrollo que permite construir aplicaciones móviles nativas utilizando JavaScript y React. A diferencia de React JS, React Native no utiliza el DOM del navegador para representar la interfaz de usuario, sino que utiliza componentes nativos de iOS y Android.

Esto significa que las aplicaciones creadas con React Native se ejecutan directamente en el dispositivo y tienen un aspecto y rendimiento similares a las aplicaciones nativas.

Con los conceptos claros acerca de qué hace cada una de estas tecnologías, podemos decir que la principal diferencia entre React JS y React Native radica en la plataforma de destino de las aplicaciones. React JS se enfoca en el desarrollo de aplicaciones web, mientras que React Native se centra en el desarrollo de aplicaciones móviles nativas. Esto significa que, si queremos construir una aplicación web, React JS es la opción adecuada, mientras que, si deseamos crear una aplicación móvil nativa, React Native es la elección ideal.

Otra diferencia importante es la forma en que se construyen las interfaces de usuario. En React JS, se utilizan elementos HTML y CSS para crear la interfaz de usuario, mientras que en React Native, se utilizan componentes nativos de iOS y Android, como vistas, texto, botones, etc. Esto permite que las aplicaciones creadas con React Native tengan una apariencia y comportamiento nativo en cada plataforma.

Figura 25: Las versiones *mobile* de Facebook e Instagram están construidas con React Native



Fuente: captura de pantalla de Google Play Store.

Qué es Expo

Expo es una plataforma y conjunto de herramientas que simplifica el proceso de desarrollo de aplicaciones móviles utilizando React Native. Al utilizar Expo, puedes crear aplicaciones móviles tanto para iOS como para Android utilizando JavaScript y React Native, sin la necesidad de configurar entornos de desarrollo específicos para cada

plataforma.

Expo se encarga de muchos aspectos del desarrollo, como la gestión de dependencias, la construcción de la aplicación y la implementación en los dispositivos. Para realizar una configuración inicial con Expo, debemos instalar el **CLI** de Expo en nuestra computadora.

Esta herramienta de línea de comandos nos permite crear y **administrar proyectos Expo**. Una vez finalizada la instalación, ya podemos utilizar el comando **expo init** para crear un nuevo proyecto.

Figura 26: Comando CLI para iniciar una aplicación de React Native



Fuente: [Imagen sin título sobre comando CLI]. (s. f.). <https://bit.ly/3NGuk1H>

Durante el proceso de inicialización, Expo solicitará que elijamos una plantilla de proyecto, entre todas las opciones predefinidas para diferentes tipos de aplicaciones, como "*blank*" (vacío), "*tabs*" (pestañas) o "*drawer*" (menú desplegable).

Creado el proyecto inicial, Expo proporciona una estructura de directorios básica y un conjunto de archivos de configuración. La estructura es similar a lo analizado oportunamente con un proyecto nuevo de React JS.

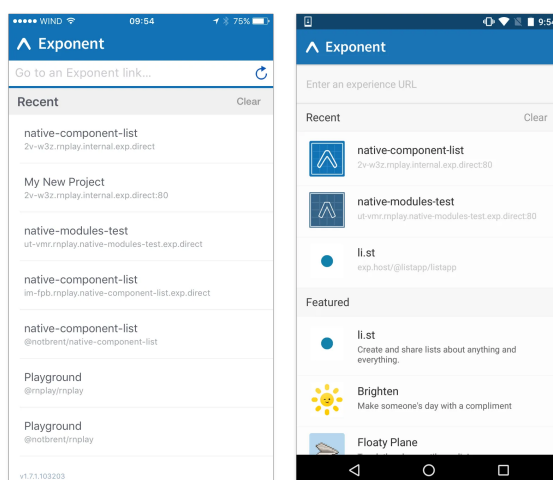
Los archivos de configuración nos permitirán personalizar la apariencia de la app, agregar dependencias y configurar otros aspectos, como ser: el acceso a permisos del dispositivo o las notificaciones *push*, entre otros.

Expo Client

Además, Expo incluye una herramienta llamada **Expo Client**, que puedes instalar en tu dispositivo móvil. Esta aplicación nos permite ver en tiempo real dentro de los dispositivos móviles los cambios que realizas en tu código, sin tener que reconstruir y volver a implementar la aplicación en el dispositivo.

Para acceder a la funcionalidad de *testing* sobre un dispositivo móvil físico, solo debemos escanear el código QR generado por Expo y veremos la *app* en acción.

Figura 27: Interfaz de configuración de Expo Client



Fuente: [Imagen sin título sobre interfaz de configuración de Expo Client]. (s. f.). <https://bit.ly/3r1WSKf>

Una vez configurado un proyecto React Native con Expo, podremos dar inicio al desarrollo de la lógica de la *app*, utilizando directamente JavaScript y React Native. Además, podremos utilizar las API nativas que nos proporciona Expo y sus componentes predefinidos, los cuales se asimilaran a las interfaces nativas de Android e iOS.

A través de las API, podremos acceder a funcionalidades del *hardware* de los dispositivos, como ser, la cámara, geolocalización, acelerómetro, almacenamiento local, etcétera.

Dimensiones fijas en componentes

Al desarrollar aplicaciones con React Native, estas estarán mayormente orientadas a dispositivos móviles, donde el tamaño de pantalla es significativamente menor, y en casi todos los casos, desplegadas en formato vertical, por lo tanto, el desarrollo de componentes gráficos debe ser pensado en ese formato. Claramente, notamos una diferencia inicial respecto al desarrollo de aplicaciones web.

Por ello, debemos pensar aquí en la elaboración de componentes React con una interfaz basada en dimensiones fijas. En React Native, contamos con la posibilidad de desarrollar componentes con dimensiones fijas. Estas refieren al establecimiento de un tamaño específico y estático para dicho componente. En otras palabras, se define una altura y ancho determinados que no cambian en función del contenido o del tamaño de la ventana del navegador.

Cuando establecemos dimensiones fijas, limitamos la flexibilidad del componente para adaptarse a diferentes tamaños de pantalla y contenido dinámico. Sin embargo, hay situaciones en las que se requiere un tamaño preciso y constante, como en elementos de diseño específicos o en componentes que deben ocupar un espacio predefinido en un diseño general.

Veamos un ejemplo de estas, a continuación.

Figura 28. Código

A screenshot of a code editor window titled "Componentes con dimensiones fijas". The code defines a functional component named "MiComponente" that returns a `<div>` element with a fixed width of 200px and height of 100px. The text "Contenido del componente" is placed inside the `<div>` tag. The component is exported as the default export.

```
import React from 'react';
const MiComponente = () => {
  return (
    <div style={{ width: '200px', height: '100px' }}>
      Contenido del componente
    </div>
  );
}

export default MiComponente;
```

Fuente: elaboración propia.

En este ejemplo anterior, el componente `<div>` tiene un tamaño fijo de **200 píxeles de ancho y 100 píxeles de alto** utilizando una definición de estilos en línea. El tamaño del

componente no cambiará, independientemente del contenido que se agregue a su interior.

También contamos con la posibilidad de definir dimensiones fijas utilizando clases CSS.

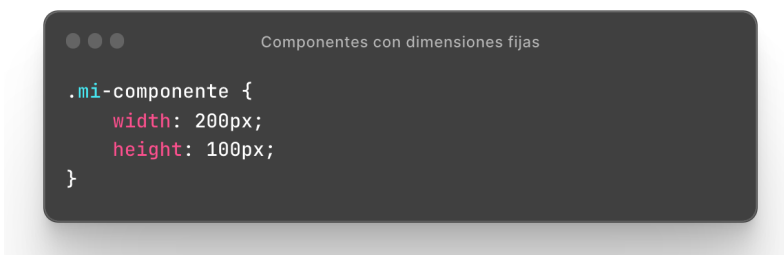
Figura 29. Código



Fuente: elaboración propia.

En el archivo CSS **MiComponente.css**, definiremos una clase con dimensiones fijas.

Figura 30. Código



Fuente: elaboración propia.

Cuando aplicamos la clase **.mi-componente** al componente `<div>`, se establecen en este último las dimensiones fijas definidas en la clase CSS.

Es importante tener en cuenta que, al utilizar dimensiones fijas, es posible que el contenido del componente se desborde o se corte si excede el espacio disponible. Por lo tanto, es recomendable utilizar dimensiones fijas en situaciones donde se pueda garantizar que el contenido no superará el tamaño especificado.

Renderizado de componentes con *flex flat* (componentes eficientes)

El manejo de renderizado de componentes mediante **Flexbox** en React JS es una técnica popular y poderosa para crear diseños flexibles y responsivos. Flexbox es el modelo de diseño de CSS que nos permite organizar y alinear elementos en un contenedor de manera dinámica y adaptable.

Tanto en React JS como utilizando React Native, podremos aplicar la propiedad **display** con el valor **flex** en los estilos de un componente para habilitar el uso de Flexbox. Al establecer el contenedor padre como **flex container**, se puede controlar la disposición y el comportamiento de los componentes hijos.

Figura 31. Código



```
import React from 'react';

const MiComponente = () => {
  return (
    <div className="contenedor-padre">
      <div className="componente1">Componente 1</div>
      <div className="componente2">Componente 2</div>
      <div className="componente3">Componente 3</div>
    </div>
  );
}
```

Fuente: elaboración propia.

En el ejemplo de código anterior, tenemos un componente llamado **MiComponente**, el cual contiene tres elementos **<div>** definidos como componentes hijos. Para aplicar **Flexbox** y manejar su renderizado, utilizamos clases CSS y estilos en línea. Veamos ahora qué estructura debemos darle a la hoja de estilos CSS que aplicamos en estos componentes.

Figura 32. Código



```
.contenedor-padre {  
  display: flex;  
  flex-direction: row;  
  justify-content: space-between;  
  align-items: center;  
}  
  
.componente1, .componente2, .componente3 {  
  flex: 1;  
  padding: 10px;  
}
```

Fuente: elaboración propia.

En este caso, la clase CSS denominada **.contenedor-padre** establece el contenedor principal como **flex container** con dirección de fila (**row**). Esto significa que los componentes hijos se colocarán uno al lado del otro en una fila.

La propiedad **.justify-content** con el valor **space-between** distribuye el espacio disponible de manera uniforme entre los componentes hijos, así crea un espacio igual entre ellos.

La propiedad **align-items** con el valor **center** alinea verticalmente los componentes hijos en el centro del contenedor.

Por último, en las clases **.componente1**, **.componente2** y **.componente3** aplicamos una flexibilidad igual (**flex: 1**) para que estas ocupen el espacio disponible de manera equitativa. Además, agregamos un espacio de relleno (**padding**) para separar visualmente los componentes entre sí.

Gracias a Flexbox, el manejo del renderizado de componentes en React JS nos permite crear diseños flexibles y responsivos que se ajustan automáticamente a diferentes tamaños de pantalla y dispositivos. Esto logrará que todo el desarrollo de la interfaz visual de React Native se base en una estructura de componentes eficientes.

Actividades de repaso

Cuando pensamos en incluir módulos JS en una aplicación, una de las cuestiones que debemos tener en cuenta para crear funciones reutilizables es que:

Cuenten con el atributo `type="module"` definido en la función.

Utilicen la cláusula `import` como parámetro entre sus paréntesis.

Estén declaradas de la forma más reutilizable posible.

Identifica dos características que nos benefician cuando importamos funciones desde un archivo JS a otro archivo JS:

Duplicar funciones con la misma lógica.

Declarar archivos JS al final de todos los documentos HTML.

Cargar en memoria código JS innecesario.

Cuando declaramos una función anteponiendo la palabra reservada `export`, logramos:

Convertir a la función en un objeto importable.

Convertir a la función en un objeto exportable.

Convertir a la función en un módulo JavaScript.

Unidad 2: Calidad de *software*

En el mundo del desarrollo de *software*, la calidad es un factor fundamental para garantizar el éxito de cualquier proyecto. La calidad de *software* se refiere a la medida en que un sistema cumple con los requisitos, expectativas y estándares establecidos. A lo largo de los años, se han desarrollado diversas metodologías y enfoques para asegurar la calidad en el proceso de desarrollo y entrega de *software*. Y, en esta última década, la demanda del mercado de profesionales orientados a la calidad del *software* crece de forma exponencial.

En esta introducción, exploraremos los conceptos de la calidad de *software*, su importancia en la satisfacción del usuario final y cómo los equipos de desarrollo se esfuerzan por alcanzar altos estándares de calidad en cada etapa del ciclo de vida del *software*.

Descubriremos cómo la calidad de *software* se ha convertido en un aspecto crítico en la industria, que garantiza la confiabilidad, seguridad y eficiencia de las aplicaciones que utilizamos en nuestra vida diaria; qué tipos de herramientas nos brindan los navegadores web modernos para aplicar el *testing* de aplicaciones web de forma rápida y efectiva sobre nuestros proyectos *frontend*.

Tema 1: *Testing* de aplicaciones web

El *testing* de aplicaciones web se enfoca en evaluar y verificar que todas las funcionalidades y componentes de una aplicación web cumplan con los requisitos y expectativas establecidos. A lo largo del tiempo, han surgido diversas metodologías y enfoques para llevar a cabo el *testing* de aplicaciones web de manera eficiente y efectiva.

En esta unidad de estudio, exploraremos la importancia del *testing* en el desarrollo de aplicaciones web, destacando su papel en la detección temprana de errores y en la mejora de la experiencia del usuario. Descubriremos cómo el *testing* de aplicaciones web se ha convertido en una práctica esencial para garantizar la calidad y el éxito de las aplicaciones en el entorno digital actual.

Cómo testear una aplicación web

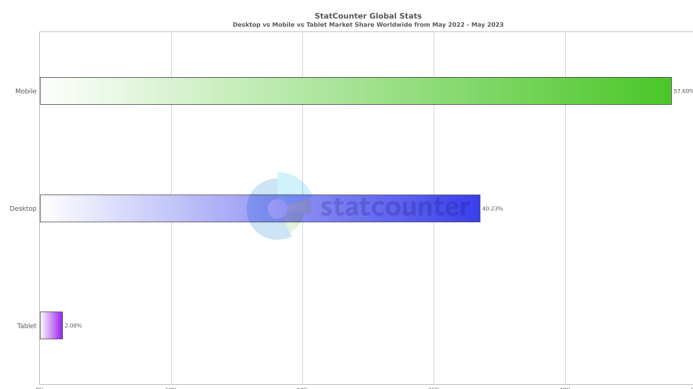
Al testear una aplicación web, llevamos a cabo una serie de actividades para verificar su correcto funcionamiento y asegurar la calidad del *software*. Nosotros realizamos pruebas en diferentes niveles, desde pruebas unitarias que evalúan componentes individuales hasta pruebas de integración que evalúan la interacción entre los diferentes componentes de la aplicación.

En nuestro proceso de testeo de una aplicación web, podemos aplicar el uso de diversas técnicas y herramientas para identificar y corregir posibles errores. Esto incluye la

ejecución de pruebas funcionales para verificar que la aplicación cumpla con los requisitos establecidos, pruebas de rendimiento para evaluar su desempeño y capacidad de respuesta, pruebas de seguridad para identificar vulnerabilidades y pruebas de compatibilidad para garantizar su correcto funcionamiento en diferentes navegadores y dispositivos.

Este último escenario no puede pasarse por alto, dado que, desde hace varios años, a nivel mundial, más del 60 % de la navegación web se realiza desde dispositivos móviles y no desde una computadora de escritorio.

Figura 33. Estadísticas de uso de la web (mobile, tablet, desktop) 2022-2023



Fuente: [Imagen sin título sobre estadísticas de uso de la web]. (s. f.). <https://bit.ly/46npPjS>

En este gráfico anterior, a nivel mundial, vemos que en las estadísticas representadas entre mediados de 2022 y mediados de 2023, los dispositivos móviles (*smartphones* y *tablets*) representan arriba del 60 % de la navegación en Internet. En el mercado argentino, este porcentaje es ligeramente mayor al promedio mundial.

Uso de *frameworks* para *testing*

Dentro del universo del *testing*, contamos también con la posibilidad de emplear *frameworks* y bibliotecas de *testing* que nos facilitan la automatización de las pruebas y nos permiten realizar pruebas repetibles y consistentes. Estas herramientas nos brindan métodos y utilidades que simplifican la escritura y ejecución de pruebas, y facilitan la detección y corrección de errores de manera más eficiente.

Existe una diversidad de *frameworks* enfocados a pruebas de aplicaciones web. A su

vez, estas se dividen en pruebas manuales y pruebas automatizadas. La oferta es muy amplia y no siempre es genérica para todo tipo de aplicaciones web *frontend*. En algunos casos, cuando la tecnología (lenguaje de programación) empleada es muy específica, podemos encontrar herramientas enfocadas en ella para permitirnos la correcta implementación de las pruebas.

Además de las pruebas manuales y pruebas automatizadas, existen otros tantos escenarios propios de acuerdo a diferentes tipos de pruebas que podemos implementar. Entre estos encontramos:

- pruebas unitarias;
- pruebas de integración;
- pruebas de extremo a extremo;
- pruebas de rendimiento;
- y más.

Pruebas unitarias

Las pruebas unitarias desempeñan un papel fundamental en el desarrollo de aplicaciones web *frontend*, ya que se enfocan en evaluar componentes individuales de forma aislada. Estas pruebas se centran en verificar el correcto funcionamiento de cada componente y su interacción con otros elementos de la aplicación.

Si bien es posible realizar pruebas unitarias de forma genérica, para cualquier tipo de aplicación web *frontend*, debemos saber también que existen en el mercado actual diferentes herramientas y *frameworks*, según la tecnología utilizada. Por ejemplo, en el caso de ReactJS, podríamos emplear a **Jest**, un popular *framework* de pruebas. Este ofrece una serie de funcionalidades que nos permiten escribir pruebas para evaluar el comportamiento y la lógica de los componentes React.

La finalidad de las pruebas unitarias es asegurarse de que cada componente funcione según lo esperado, identificar y solucionar posibles errores y mantener un código limpio

y robusto. Al realizar estas pruebas de forma regular durante el proceso de desarrollo de la aplicación web, se incrementa la confianza en la calidad del software y se facilita la detección temprana de problemas, lo que resultará en un proceso de desarrollo más eficiente y efectivo.

Figura 34. Unit Testing para comprobar el correcto funcionamiento de fragmentos de código



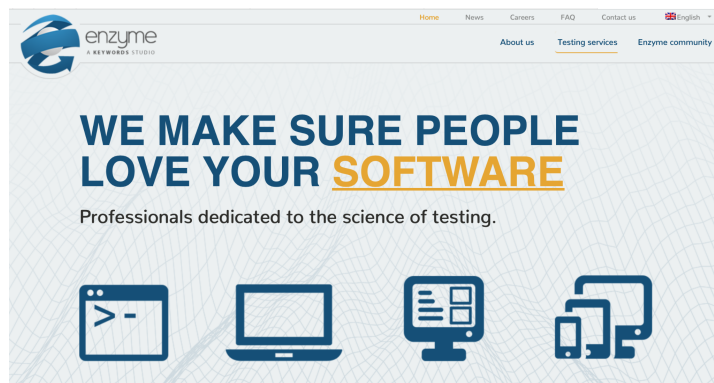
Fuente: [Imagen sin título sobre Unit Testing]. (s. f.). <https://bit.ly/3CJQVUR>

Pruebas de integración

Además de las pruebas unitarias, consideramos de gran importancia realizar pruebas de integración en nuestras aplicaciones web. Estas pruebas se enfocan en evaluar la interacción y la integración entre los distintos componentes y módulos de la aplicación, para brindarnos mayor confianza en su funcionamiento global.

En nuestro enfoque de pruebas de integración, utilizamos diversas herramientas y técnicas que nos permiten simular eventos y manipular el estado de los componentes para evaluar su interacción. Estas pruebas nos ayudan a detectar posibles problemas de comunicación entre los distintos elementos de la aplicación y asegurar que trabajen de manera cohesiva.

Figura 35. Enzyme ofrece herramientas para *testing* para cualquier segmento



Fuente: captura de pantalla del sitio web enzyme.

Al realizar pruebas de integración, podemos identificar posibles errores en la comunicación entre los componentes, asegurar una correcta transferencia de datos y validar el flujo de la aplicación en su conjunto. Esto nos permite mantener una alta calidad en nuestras aplicaciones web y brindar una experiencia fluida y satisfactoria para los usuarios.

Pruebas de extremo a extremo

Las pruebas de extremo a extremo, pruebas *end-to-end*, se centran en simular y evaluar escenarios de uso reales, desde el inicio hasta el final de la aplicación, con el objetivo de verificar su funcionamiento global y su interacción con diferentes componentes y sistemas externos. Durante este tipo de *testing*, se busca simular acciones y eventos que los usuarios realizan en la aplicación web, como hacer clic en botones, ingresar datos en formularios, navegar distintas páginas o secciones. De esta forma, podremos identificar posibles problemas de flujo, funcionalidad y rendimiento, así como también validar la correcta integración de todos los elementos de la aplicación.

Estas pruebas nos garantizan que la aplicación se comporte de manera esperada y proporcione una experiencia de usuario satisfactoria. Gracias a una evaluación del *software* en su conjunto, lograremos detectar posibles errores, inconsistencias y vulnerabilidades que podrían pasar desapercibidas en pruebas más centradas en partes independientes.

De esta manera, lograremos detectar y corregir problemas antes de que afecten a los usuarios finales y asegurar el correcto funcionamiento de la aplicación en diferentes escenarios y condiciones. Este tipo de pruebas desempeñan un papel fundamental en el

proceso de desarrollo y contribuyen a la entrega de aplicaciones web *frontend* sólidas y confiables.

Pruebas de rendimiento

Este tipo de pruebas nos permite evaluar cómo se comporta nuestra aplicación en términos de velocidad, capacidad de respuesta y eficiencia en diferentes escenarios y cargas de trabajo. Durante las pruebas de rendimiento, analizamos el desempeño de la aplicación en situaciones de alto tráfico, cargas de datos voluminosas y diversas condiciones de red. A través de herramientas especializadas, medimos y registramos métricas clave, como el tiempo de carga de la página, la capacidad de respuesta del usuario y la utilización de recursos del sistema.

De esta forma, podremos identificar cuellos de botella, puntos de debilidad y oportunidades de optimización en la aplicación web. Evaluando su rendimiento, podemos detectar posibles retrasos, bloqueos, fugas de memoria u otras limitaciones que puedan afectar negativamente la experiencia del usuario. De este tipo de pruebas surgirán posibles mejoras enfocadas:

- en optimización del código;
- agregar técnicas de almacenamiento en *caché*;
- optimizar y reducir el tamaño de los recursos descargados;
- mejorar consultas a bb.dd del lado del *backend*.

Y otras tantas estrategias más.

Figura 36. El uso de pruebas de rendimiento es ideal para momentos específicos, donde la demanda de acceso es alta y continua



Fuente: [Imagen sin título sobre black friday]. (s. f.). <https://bit.ly/3XqwohO>

Testing automatizado versus testing manual

En el caso de las aplicaciones web *frontend*, existen dos enfoques principales que utilizamos: el *testing* automatizado y el *testing* manual. Cada enfoque tiene sus propias fortalezas y debilidades, y como equipo de desarrollo, es importante comprender cuándo y cómo aplicar cada uno de ellos.

En última instancia, el objetivo de combinar ambos enfoques es lograr un nivel óptimo de calidad en nuestra aplicación. El *testing* automatizado nos brinda una base sólida para validar rápidamente la funcionalidad y detectar errores, mientras que el *testing* manual nos permite poner a prueba la aplicación desde la perspectiva de un usuario real y mejorar aspectos subjetivos de la experiencia.

Como equipo de desarrollo, es fundamental establecer un equilibrio entre el *testing* automatizado y el *testing* manual, para adaptarnos a las necesidades y características específicas de nuestro proyecto. De esta manera, podemos garantizar que nuestra aplicación web *frontend* cumpla con los estándares de calidad y brinde una experiencia excepcional a nuestros usuarios.

Testing automatizado

En nuestras pruebas automatizadas, utilizamos herramientas y *frameworks* especializados para escribir casos de prueba que se ejecutan automáticamente. Estas pruebas nos permiten validar rápidamente la funcionalidad de nuestra aplicación y

asegurarnos de que todo funcione correctamente. Además, nos brindan la capacidad de detectar regresiones y errores de manera eficiente, especialmente, cuando realizamos cambios en nuestro código o añadimos nuevas funcionalidades.

Algunas ventajas del *testing* automatizado son las siguientes:

Eficiencia y ahorro de tiempo: las pruebas automatizadas se ejecutan rápidamente y pueden cubrir una amplia gama de casos de prueba en poco tiempo. Esto ahorra tiempo y recursos en comparación con el *testing* manual, especialmente, en proyectos grandes y complejos.

Mayor cobertura: con las pruebas automatizadas, es posible lograr una mayor cobertura de pruebas al ejecutar casos de prueba repetitivos, *edge cases* y escenarios complejos de manera consistente. Esto ayuda a identificar y corregir problemas de forma más efectiva.

Reutilización y mantenimiento: las pruebas automatizadas se pueden reutilizar fácilmente a medida que se realizan cambios en el código. Esto facilita el mantenimiento y evita la repetición de esfuerzos al ejecutar pruebas en diferentes etapas del desarrollo.

Integración continua: el *testing* automatizado se puede integrar fácilmente en un flujo de trabajo de integración continua (CI) para ejecutar pruebas automáticamente después de cada cambio en el código fuente. Esto permite identificar problemas rápidamente y garantizar la estabilidad del proyecto en todo momento.

***Testing* manual**

Por otro lado, el *testing* manual también desempeña un papel crucial en nuestro proceso de desarrollo. Con este enfoque, dedicamos tiempo y esfuerzo a probar nuestra aplicación de forma interactiva, explorando diferentes escenarios y comprobando el flujo de trabajo en busca de posibles problemas.

A través de este tipo de *testing*, podemos identificar aspectos como la usabilidad, la accesibilidad y la experiencia del usuario, los cuales, a menudo, no son fáciles de capturar en pruebas automatizadas. Su modalidad de trabajo es más laboriosa, en comparación con el *testing* automatizado. Entre sus ventajas, encontramos:

Exploración exhaustiva: como *testers* podemos explorar la aplicación en busca de problemas no planificados y realizar pruebas *ad hoc* para verificar diferentes escenarios de uso. Esto ayudará a descubrir errores y comportamientos inesperados que podrían haber pasado desapercibidos en pruebas automatizadas.

Experiencia de usuario real: el *testing* manual nos permite evaluar la experiencia de usuario real al interactuar directamente con la interfaz de la aplicación. Como *testers* podremos identificar posibles problemas de usabilidad, diseño o accesibilidad que podrían afectar la satisfacción del usuario.

Casos de prueba complejos: en algunos casos, las pruebas automatizadas pueden resultar difíciles o costosas de implementar para casos de prueba particularmente complejos o específicos. El *testing* manual puede ser más flexible y adaptarse mejor a estas situaciones.

Validación visual: el *testing* manual permite una validación visual detallada de la interfaz de usuario y los elementos visuales de la aplicación. Como *testers* manuales, podremos verificar la apariencia, la disposición y la consistencia visual en diferentes dispositivos y tamaños de pantalla.

Ventajas y desventajas de automatizar pruebas

Entre las ventajas de utilizar pruebas automatizadas, destacamos:

- Eficiencia y ahorro de tiempo.
- Mayor cobertura de pruebas.
- Reutilización y mantenimiento.
- Integración continua.

Y como desventajas, no podemos dejar de mencionar:

- Costo inicial y complejidad.
- Limitaciones en casos de pruebas complejos.
- El mantenimiento de las pruebas.
- Limitaciones en experiencia como usuarios finales.

Herramientas de *software* para automatización

Dentro del compendio de *softwares* existentes para automatizar pruebas, podemos destacar a las siguientes herramientas como las más utilizadas en la actualidad.

Selenium: una de las herramientas más conocidas y utilizadas para la automatización de pruebas web. Permite interactuar con los elementos de la interfaz de usuario y ejecutar pruebas en diferentes navegadores.

Cypress: una herramienta de automatización de pruebas moderna y de código abierto. Proporciona una API intuitiva y potente para escribir pruebas funcionales y de integración de manera sencilla.

Puppeteer: una biblioteca de Node.js desarrollada por Google. Permite controlar un navegador de forma programática y automatizar acciones como navegación, clics y capturas de pantalla.

TestCafe: una herramienta de automatización de pruebas diseñada para pruebas de extremo a extremo. Permite ejecutar pruebas en múltiples navegadores sin necesidad de configuración adicional.

Playwright: una biblioteca de automatización de pruebas desarrollada por Microsoft. Permite automatizar pruebas en diferentes navegadores, dispositivos y plataformas.

Appium: una herramienta de automatización de pruebas móviles que también puede ser utilizada para pruebas web. Permite interactuar con aplicaciones web en dispositivos móviles y emuladores.

WebDriverIO: un *framework* de automatización de pruebas basado en Selenium. Proporciona una API sencilla y potente para escribir pruebas en JavaScript y ejecutarlas en diferentes navegadores.

Dado que existe un compendio de diferentes tecnologías para desarrollar aplicaciones web, no hay una gran variedad de opciones de utilizar herramientas genéricas para todo. En general, cada tecnología cuenta con sus herramientas de *testing* aplicadas, dado que permitirán profundizar en la automatización de la mano de la tecnología elegida para construir el *software* en cuestión.

Actividades de repaso

Identificar, a continuación, una de las características principales de los componentes reutilizables que ofrece React:

Crear una interfaz de desarrollo similar a la que propone Vanilla JavaScript.

Facilitar la construcción y mantenimiento de interfaces de usuario complejas.

Evitar un uso repetido de HTML en diferentes partes de la aplicación.

Cuando hablamos de React, podemos definir a esta herramienta como:

Un framework progresivo y de fácil aprendizaje.

Un framework complementario a Node.js para desarrollar aplicaciones backend.

Una librería JavaScript de código abierto, performante y modular.

Identifica dos características que aplican a las librerías JS:

Podemos utilizarlas de forma independiente, según sea necesario.

Imponen una forma de trabajo sobre una estructura de código rígida.

Ofrecen funcionalidades específicas, como manipular el DOM y llamadas a API.

Tema 2: Estrategias de *testing* automatizado

La selección adecuada de casos de prueba es esencial para garantizar una cobertura completa y efectiva al realizar pruebas en aplicaciones web *frontend*.

Con una amplia variedad de componentes y funcionalidades en nuestras aplicaciones, es fundamental seleccionar cuidadosamente los casos de prueba para garantizar la calidad y el rendimiento. En esta unidad de estudio, nos enfocaremos en explorar algunas consideraciones importantes para una correcta selección de casos de prueba.

Identificación de escenarios clave

El primer paso en la selección de casos de prueba es identificar los escenarios clave que deben probarse. Esto implica comprender los requisitos del proyecto y las funcionalidades críticas de la aplicación. Los casos de prueba deben cubrir tanto los flujos principales como los casos excepcionales para asegurar que todas las partes importantes de la aplicación se prueben exhaustivamente.

Pensando en un escenario de una aplicación web orientada a *ecommerce*, uno de los puntos clave que debemos identificar aquí es el de validar a los usuarios mediante procesos de *login*, como también aquellos otros procesos puntuales de las pasarelas de pago.

En ambos escenarios, la seguridad es lo que más exhaustivamente debemos probar, para garantizar la integridad de la información que se maneja dentro de estos procesos.

Priorizar casos de prueba

No todos los casos de prueba tienen la misma prioridad. Es importante establecer una jerarquía y priorizar los casos de prueba en función de la criticidad y el impacto potencial de los errores. Los casos de prueba que cubren las funcionalidades más importantes, los escenarios de alto riesgo y los flujos críticos deben recibir una prioridad más alta durante la selección y ejecución de las pruebas.

Cobertura de código

Al seleccionar casos de prueba, es crucial tener en cuenta la cobertura de código. La cobertura de código se refiere a qué porcentaje del código fuente está siendo ejecutado por las pruebas. Es importante seleccionar casos de prueba que ayuden a alcanzar una cobertura óptima, especialmente, en áreas críticas o complejas del código.

Las herramientas de prueba pueden proporcionar información sobre la cobertura de código y ayudar a identificar áreas que requieren una mayor atención.

Pruebas de integración

En nuestras aplicaciones, los diferentes componentes interactúan entre sí para formar la aplicación completa. Por lo tanto, es importante seleccionar casos de prueba que cubran la interacción y la integración de los componentes.

Las pruebas de integración garantizan que los diferentes componentes funcionen correctamente juntos y que los datos se transfieran adecuadamente entre ellos. Esto ayuda a evitar problemas en el flujo de datos y la comunicación entre componentes.

Validación de entradas y salidas

Un aspecto crucial de la selección de casos de prueba es validar las entradas y las salidas de los componentes. Esto implica probar diferentes tipos de datos, tamaños de datos y escenarios de entrada, para garantizar que el componente maneje correctamente todas las situaciones. Además, es importante verificar que las salidas del componente sean correctas y cumplan con los requisitos establecidos.

Pruebas de rendimiento

No se debe pasar por alto la selección de casos de prueba relacionados con el rendimiento. Las pruebas de rendimiento evalúan cómo responde la aplicación en términos de velocidad, carga de datos y utilización de recursos.

Al seleccionar casos de prueba de rendimiento, es importante considerar diferentes cargas de trabajo y escenarios de uso para identificar posibles cuellos de botella y optimizar el rendimiento de la aplicación.

La selección adecuada de casos de prueba es esencial para garantizar una cobertura completa y efectiva de la aplicación. Al identificar los escenarios clave, priorizar los casos de prueba, tener en cuenta la cobertura de código, realizar pruebas de integración, validar entradas y salidas y realizar pruebas de rendimiento, se puede lograr una selección sólida de casos de prueba. Esto ayuda a mejorar la calidad, la estabilidad y la eficiencia de las aplicaciones web desarrolladas.

Y, aunque este terreno tiene un nicho de especialistas, no está mal que nosotros, como desarrolladores de aplicaciones, entendamos de qué se trata y podamos aportar, durante el desarrollo, algunas pruebas mínimas para que la calidad del producto sea la más efectiva desde el momento en que se inicia su construcción.

Diseño de *scripts*

El diseño de *scripts* es crucial para lograr un código claro y legible. Si bien la enumeración de elementos puede ser útil en algunos casos, existen enfoques alternativos que promueven una estructura más clara y comprensible sin depender de la enumeración. A continuación, exploraremos algunas técnicas efectivas para diseñar *scripts* sin numerar elementos, lo que mejorará la legibilidad y facilitará el mantenimiento del código.

Comentarios descriptivos

En lugar de enumerar elementos, es recomendable utilizar comentarios descriptivos en el código. Los comentarios proporcionan información contextual y explican la

funcionalidad de cada elemento o bloque de código.

Al utilizar comentarios descriptivos y significativos, se facilita la comprensión del código y se evita la necesidad de depender de números de enumeración.

Figura 37. Código



Fuente: elaboración propia.

En este ejemplo de código vemos aplicado el uso de comentarios que describen de forma concisa el proceso que realiza el bloque de código contiguo. Este modelo representa una buena práctica, evita tener que extenderse por demás en la redacción descriptiva y también evita comentar cada una de las líneas de código que componen este proceso.

Indentación adecuada

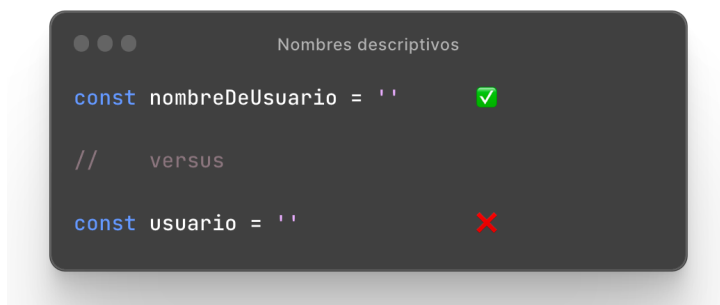
La indentación correcta es esencial para un diseño de *script* limpio y legible. Al estructurar el código con una indentación consistente, se crea una jerarquía visual clara que facilita la comprensión de la lógica y la estructura del *script*. Una correcta indentación también ayuda a distinguir los bloques de código, sin necesidad de recurrir a la enumeración de elementos.

Nombres descriptivos para variables y funciones

Otra forma de mejorar la legibilidad de un *script* sin recurrir a la enumeración es utilizar nombres descriptivos para las variables y las funciones. Asignar nombres que reflejen claramente el propósito y la funcionalidad de cada elemento del *script* ayuda a

comprender el código y reduce la necesidad de enumerar elementos para seguir la lógica.

Figura 38. Otro ejemplo que representa a los nombres descriptivos



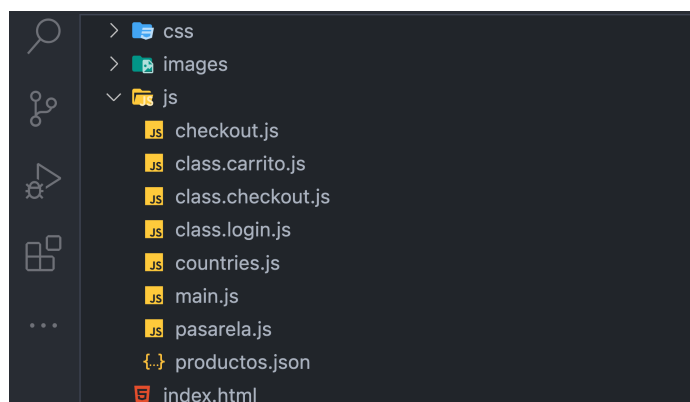
Fuente: elaboración propia.

La variable **usuario** aplica a un término mucho más generalizado que el que representa la variable **nombreDeUsuario**.

Modularización del código

La división del *script* en funciones o módulos coherentes también contribuye a un diseño claro y legible. Al dividir el código en unidades lógicas y reutilizables, se mejora la modularidad y se facilita el mantenimiento. Esto permite organizar el *script* en bloques más pequeños y autónomos, sin necesidad de recurrir a la enumeración para diferenciar entre secciones.

Figura 39. Código



Fuente: elaboración propia.

Separar las clases u objetos, diferenciar la lógica enfocada a un documento HTML principal de un documento HTML enfocado en el proceso de finalizar una compra, y definir archivos específicos que manejan pequeños procesos aislados es mucho más cómodo que tener uno o dos archivos con cientos o miles de líneas de código, que concentren todo esto en un único punto de acceso. Esto último se hace inmantenible con el tiempo y quita la posibilidad de identificar rápidamente clases u objetos que puedan ser reutilizables en otros proyectos web, diferentes al que tiene implementadas las clases en cuestión.

Uso de espacios en blanco y líneas vacías

El uso adecuado de espacios en blanco y líneas vacías en el *script* también ayuda a mejorar la legibilidad y la estructura del código. Al utilizar espacios en blanco apropiados entre secciones de código relacionadas, se facilita la identificación visual de los bloques, de manera que se elimina la necesidad de enumerar elementos para diferenciarlos.

Figura 40. Código

```
const productos = []

const tablaProductos = document.querySelector('tbody')

const retornoFilaTablaHTML = (producto) => {
  return `<tr>
    <td class="class-table-number">${producto.codigo}</td>
    <td class="emoji-image">${producto.imagen}</td>
    <td>${producto.descripcion}</td>
    <td>${producto.importe}</td>
    <td>button id="${producto.codigo}" class="button button-outline button-big-emoji"></button></td>
  </tr>`
}

const activarClickEnBotonesProductos = () => {
  const botonesProductos = document.querySelectorAll('button.button.button-outline.button-big-emoji')
  if (botonesProductos.length > 0) {
    for (boton of botonesProductos) {
      boton.addEventListener('click', (e) => {
        let productoSeleccionado = productos.find((producto) => producto.codigo === parseInt(e.target.id))
        console.log(productoSeleccionado)
      })
    }
  }
}

const cargarProductos = () => {
  let productosHTML = ''
  if (productos.length > 0) {
    productos.forEach((producto) => productosHTML += retornoFilaTablaHTML(producto))
    tablaProductos.innerHTML = productosHTML
    activarClickEnBotonesProductos()
  }
}

function obtenerProductos() {
  fetch('js/productos.json')
    .then((response) => response.json())
    .then((data) => productos.push(...data))
    .then(() => cargarProductos())
}

obtenerProductos()
```

Fuente: elaboración propia.

Este es un ejemplo de código simple, sencillo, correctamente indentado y ordenado, y que no excede su contenido con comentarios de funcionalidades ni espacios innecesarios en la codificación de su lógica.

Adherencia a convenciones de estilo de código

Seguir convenciones de estilo de código establecidas también ayuda a mejorar la legibilidad del *script*. Estas convenciones incluyen el uso consistente de mayúsculas y minúsculas, la alineación adecuada de los elementos y la organización coherente del código. Al seguir convenciones de estilo, se logra un código más uniforme y comprensible sin depender de la enumeración de elementos. ¿Nombre de variables y constantes en inglés? ¿En español? ¿Uso la nomenclatura *camel-case* o *snake-case*?, entre otras tantas convenciones.

Monitoreo y mantenimiento de pruebas automatizadas

Las pruebas automatizadas desempeñan un papel crucial en nuestro proceso de desarrollo de aplicaciones web *frontend*, al garantizar la calidad y el rendimiento del *software*. Sin embargo, es importante reconocer que estas pruebas también requieren monitoreo y mantenimiento continuos para asegurar su efectividad a lo largo del tiempo.

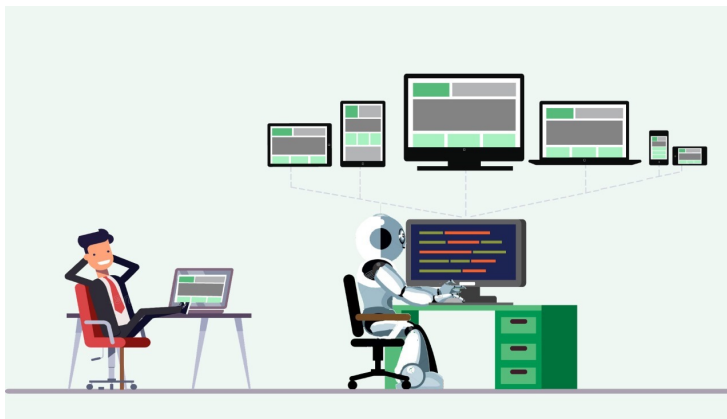
A continuación, exploraremos la importancia del monitoreo y mantenimiento de pruebas automatizadas y analizaremos algunas buenas prácticas para su implementación.

Monitoreo de las pruebas automatizadas

Es fundamental establecer un sistema de monitoreo de pruebas automatizadas para identificar cualquier problema o fallo en el proceso de prueba. Esto implica monitorear regularmente la ejecución de las pruebas para asegurarnos de que se están ejecutando correctamente y proporcionando resultados precisos.

El monitoreo también puede incluir la supervisión de tiempos de ejecución, errores recurrentes y cambios en el comportamiento esperado de la aplicación.

Figura 41. Monitoreo de test automatizados



Fuente: [Imagen sin título sobre monitoreo de test automatizados]. (s. f.). <https://bit.ly/3peeJNE>

Actualización de pruebas automatizadas

A medida que nuestra aplicación web evoluciona y se actualiza, es necesario actualizar y mantener las pruebas automatizadas para que sigan siendo relevantes y efectivas. Esto implica revisar y modificar los *scripts* de prueba existentes para reflejar los cambios en la funcionalidad de la aplicación. También es importante agregar nuevas pruebas o ajustar las existentes para cubrir nuevas características o casos de uso.

Tampoco podemos descuidar la implementación de actualizaciones de los casos de prueba cuando se integran nuevos módulos a módulos existentes y, entre estos, comparten la generación y actualización de datos. La integridad y regresión deberá sumarse a la actualización de las pruebas automatizadas.

Mantenimiento de los datos de prueba

El mantenimiento adecuado de los datos de prueba es esencial para garantizar la validez y la coherencia de las pruebas automatizadas. Esto incluye asegurarnos de que los conjuntos de datos de prueba estén actualizados y sean representativos de los datos reales con los que se encontrará la aplicación en producción. Además, debemos realizar ajustes a los datos de prueba en caso de cambios en la estructura de la aplicación o en los requisitos de prueba.

Revisión periódica de informes de prueba

Los informes de prueba generados por las pruebas automatizadas en nuestras aplicaciones web *frontend* proporcionan información valiosa sobre el rendimiento y la estabilidad del *software*. Es fundamental que revisemos estos informes de manera regular para identificar patrones, tendencias y problemas recurrentes. De esta manera, podremos tomar medidas correctivas de forma oportuna y mejorar la efectividad de nuestras pruebas automatizadas en general.

Integración con nuestro flujo de trabajo de desarrollo

Para garantizar un mantenimiento y monitoreo efectivos, es recomendable integrar las pruebas automatizadas en nuestro flujo de trabajo de desarrollo, lo que implicará ejecutar regularmente las pruebas automatizadas como parte de nuestro proceso de integración continua (CI) y aprovechar herramientas de **CI/CD (integración continua/entrega continua)** para automatizar la ejecución y generación de informes de pruebas. Esta integración nos facilitará el monitoreo y el mantenimiento continuo de nuestras pruebas automatizadas.

Actividades de repaso

Identifica, a continuación, dos características aplicables en el testing automatizado:

Actualización de pruebas automatizadas.

Aplicar la calidad del software en el testing.

Revisión de los informes de prueba.

Ante un escenario de software orientado a un ecommerce, identifica dos puntos clave que se van a evaluar durante el testing de una aplicación de software con estas características:

Integridad y no repetición de productos cargados en un carrito.

Proceso de inicio de sesión del usuario.

Manejo de pasarelas de pago.

En el escenario del testing automatizado, no se deben tener en cuenta las buenas prácticas en el código desarrollado, como ser: nombres de variables, convenciones ni tampoco la correcta indentación del código escrito.

Es una afirmación falsa.

Es una afirmación verdadera.

Esta afirmación varía según el tester.

Tema 3: Selenium WebDriver

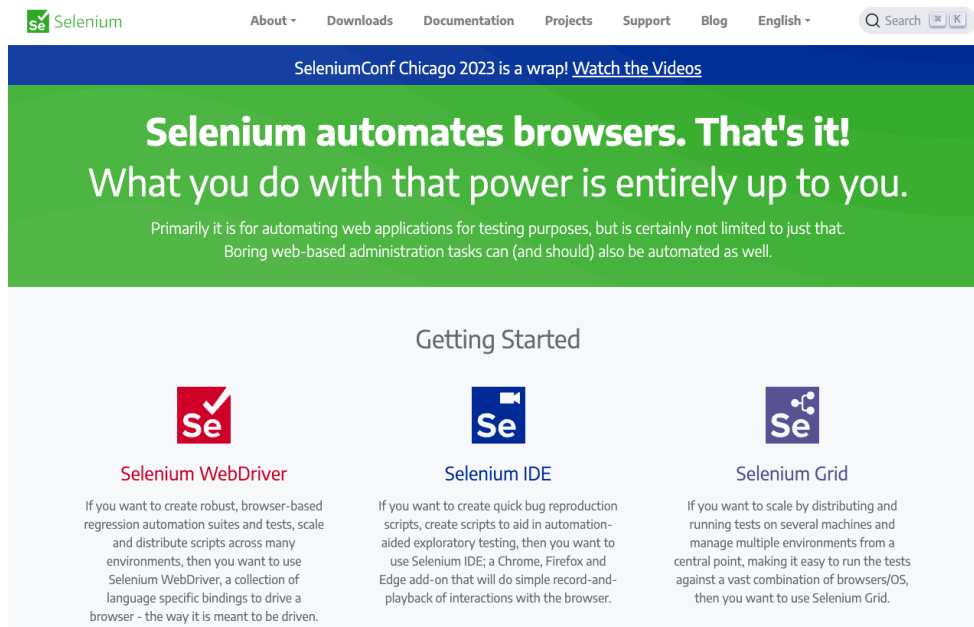
En el terreno del *testing* automatizado, entre todas las herramientas de *software* que complementan estas tareas de análisis y búsqueda de errores, existe una con una alta popularidad en el segmento de la calidad de *software*: Selenium.

Veamos, a continuación, de qué trata esta herramienta y cómo nos puede ayudar a desarrollar pruebas de usuario automatizadas, de una forma eficaz.

Selenium WebDriver

Selenium WebDriver es una biblioteca de código abierto que forma parte del conjunto de herramientas de Selenium. Se utiliza específicamente para la automatización de pruebas en aplicaciones web. Esta herramienta permite a desarrolladores escribir *scripts* de prueba en diferentes lenguajes de programación, como Java, Python, C#, Ruby, entre otros tantos, para interactuar con los elementos y las funciones de una aplicación web.

Figura 42. Código



Fuente: captura de pantalla de web oficial de Selenium (www.selenium.dev).

Características y funcionalidades

Cuando se trata de automatizar pruebas en nuestras aplicaciones web, contamos con una amplia gama de características y funcionalidades que nos ofrece Selenium WebDriver. Algunas de las principales características son las siguientes.

Control del navegador: podemos abrir y controlar el navegador web de forma programática, lo que nos permite realizar acciones como la navegación en páginas web, el llenado de formularios, la interacción con elementos, la selección de elementos y el manejo de ventanas emergentes, entre otros.

Localización de elementos: con Selenium WebDriver, podemos localizar e interactuar con elementos en una página web utilizando diferentes selectores, como ID, clase, nombre, etiqueta, Xpath y CSS. Esto nos permite realizar acciones tales como hacer clic en botones, escribir texto en campos de entrada y verificar contenido.

Acciones del usuario: Selenium WebDriver nos proporciona métodos para simular acciones del usuario, como hacer clic en elementos, enviar teclas, mover el *mouse* y desplazarse por la página. Esto nos permite replicar interacciones humanas y probar diferentes flujos de usuario en nuestra aplicación web.

Captura de pantalla y generación de informes: Selenium WebDriver nos permite capturar imágenes de la pantalla durante la ejecución de las pruebas. Esto es útil para generar informes y depurar problemas. Además, podemos generar informes detallados de las pruebas automatizadas para analizar los resultados y tomar medidas correctivas.

Beneficios de utilizar Selenium WebDriver

Al utilizar Selenium WebDriver, obtenemos varias ventajas en la automatización de pruebas en nuestras aplicaciones web.

Multiplataforma: Selenium WebDriver es compatible con diferentes sistemas operativos, navegadores y lenguajes de programación, lo que nos brinda flexibilidad y portabilidad en la automatización de pruebas.

Amplia comunidad y soporte: contamos con una comunidad activa de usuarios y desarrolladores de Selenium WebDriver que comparten conocimientos, consejos y soluciones a través de foros, *blogs* y recursos en línea. Esto nos asegura un buen soporte y ayuda en caso de problemas o consultas.

Integración con herramientas de prueba y CI/CD: Selenium WebDriver se integra fácilmente con otras herramientas y *frameworks* de prueba, así como con flujos de trabajo de integración continua (CI) y entrega continua (CD). Esto facilita la incorporación de nuestras pruebas automatizadas en el proceso de desarrollo.

Compatibilidad con múltiples navegadores: tiene la capacidad de trabajar con varios navegadores web, incluyendo Google Chrome, Mozilla Firefox, Microsoft Edge y más. Esto nos permite realizar pruebas de compatibilidad para garantizar que nuestra aplicación se comporte correctamente en escenarios *cross-browser*.

Como podemos apreciar, Selenium WebDriver es una herramienta poderosa para la automatización de pruebas en aplicaciones web. Su capacidad para interactuar con los navegadores web y simular acciones del usuario la convierte en una elección popular

para automatizar pruebas funcionales, de rendimiento y de compatibilidad en diferentes entornos. Al utilizar Selenium WebDriver, mejoramos la eficiencia, la calidad y la estabilidad de nuestras aplicaciones web a través de pruebas automatizadas más rápidas y confiables.

Grabar y reproducir un caso de pruebas

Grabar y reproducir un caso de pruebas es una técnica fundamental en nuestra estrategia de automatización de pruebas para aplicaciones web *frontend*. Nos permite capturar las interacciones del usuario y luego reproducirlas de forma automatizada.

1. Luego de haber instalado y configurado nuestro entorno para comenzar a grabar el caso de prueba, nos queda preparar nuestra aplicación web *frontend* para la grabación.
2. Durante la grabación, realizamos una serie de acciones que representan el flujo de usuario que queremos automatizar. Esto puede incluir hacer clic en botones, completar formularios, navegar entre páginas y más.
3. Una vez grabado el caso de pruebas, procedemos a editar y refinar el *script* resultante. Eliminamos pasos innecesarios, agregamos aserciones y ajustamos parámetros o valores, según sea necesario. Es importante revisar y validar el *script* para asegurarnos de que refleje correctamente el caso de prueba deseado.
4. Con el *script* grabado y refinado, utilizamos la herramienta de reproducción para ejecutar automáticamente el caso de prueba. La herramienta reproducirá las interacciones grabadas y generará resultados basados en las aserciones y condiciones establecidas en el *script*. De esta manera, podemos verificar si nuestra aplicación se comporta correctamente y cumple con los requisitos esperados.

Es importante tener en cuenta que los casos de prueba grabados pueden requerir mantenimiento y actualización a medida que nuestra aplicación evoluciona y se actualiza.

Es posible que tengamos que ajustar el *script* para reflejar cambios en la interfaz de usuario, en la estructura del código o en los flujos de usuario. Realizar un mantenimiento regular nos asegura que nuestros casos de prueba grabados sigan siendo válidos y

efectivos a lo largo del tiempo.

La técnica de grabar y reproducir nos ofrece una forma rápida y conveniente de automatizar casos de prueba en nuestras aplicaciones web *frontend*. Al utilizar herramientas de grabación y reproducción, simplificamos el proceso de automatización y aceleramos la implementación de pruebas automatizadas. Sin embargo, es importante tener en cuenta que los casos de prueba grabados pueden requerir ajustes y mantenimiento a medida que nuestra aplicación evoluciona.

Actividades de repaso

Identifica dos características notables que encontramos al implementar Selenium WebDriver en la grabación de test de pruebas:

Permite escribir test de prueba con diferentes lenguajes de programación.

Es ideal para automatizar el testing de aplicaciones web.

Complementa al lenguaje de programación para escribir código funcional.

Menciona una ventaja en la automatización de pruebas de aplicaciones web que brinda Selenium Webdriver.

Crea los scripts de prueba automáticamente sin necesidad de instrucciones.

Se integra en el editor de código para corregir errores y bugs detectados.

Se integra para interactuar con elementos y funciones de una aplicación web.

Una vez que grabamos el caso de pruebas, el paso siguiente será:

Bloquear en modo solo lectura el script resultante y los parámetros y valores.

Editar y refinar el script resultante eliminando pasos innecesarios.

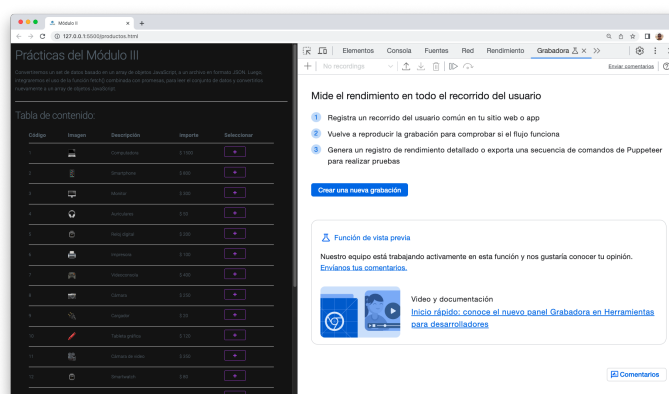
Priorizar en el script resultante aquellos pasos innecesarios de ejecución.

Tema 4: Dev Tools *recording panel*

En resumen, el panel de grabación de DevTools es una herramienta poderosa que forma parte de las herramientas de desarrollo de los navegadores web y facilita la grabación y reproducción de interacciones del usuario en una página web.

Esta característica es de gran utilidad para el desarrollo y la automatización de pruebas, permite mejorar la eficiencia y la calidad de las pruebas automatizadas en el contexto del desarrollo de aplicaciones web.

Figura 43. Dev Tools *Recording Panel* (Grabadora)



Fuente: captura de pantalla de la aplicación DevTools.

Utilizar DevTools para la realización de pruebas

Como desarrolladores web, debemos saber que el panel de grabación de DevTools no está diseñado específicamente para la automatización de pruebas, sino más bien como una herramienta de desarrollo y depuración. Aun así, podemos utilizar el panel de grabación para registrar interacciones que realizaría un usuario en nuestras aplicaciones web y generar con estas diferentes *scripts* de prueba.

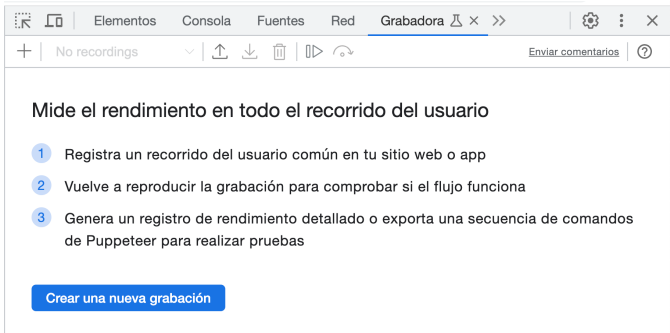
Luego, estos *scripts* pueden ser ajustados y refinados para adaptarse a un flujo de prueba automatizado más robusto y escalable. Como este panel está disponible en los

navegadores web Chrome y Edge, es ideal que podamos sacar provecho de ellos y definir un espacio de pruebas durante el desarrollo de nuestras soluciones de *software*.

Grabar un caso de prueba

Para poder grabar un caso de prueba, cargamos la aplicación web en una ventana del navegador web (Chrome o Edge), activamos **DevTools > Grabadora (Recorder**, si tienes la versión en inglés).

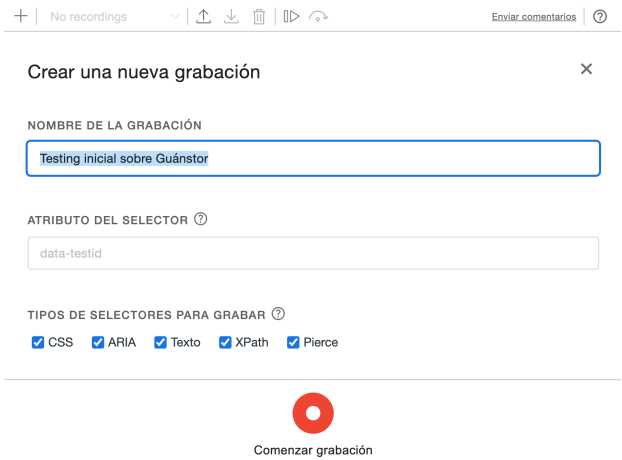
Figura 44. Grabadora en aplicación DevTools



Fuente: captura de pantalla de la aplicación DevTools.

Pulsando el botón **Crear una nueva grabación**, definimos a continuación un nombre efectivo para la grabación y ya podemos comenzar a grabar pulsando el botón rojo ubicado en la parte inferior del panel.

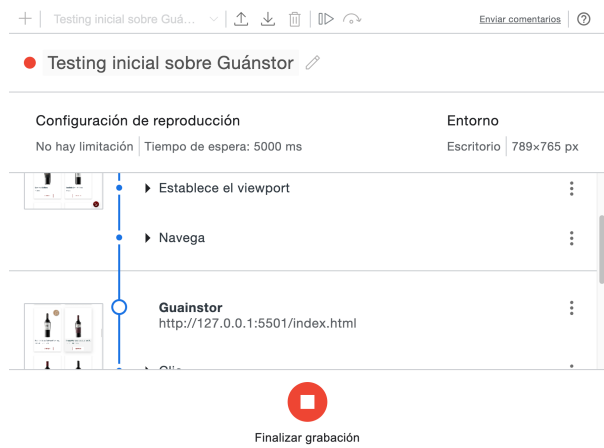
Figura 45. Creación de una nueva grabación



Fuente: captura de pantalla de la aplicación DevTools.

Así, daremos inicio al proceso en cuestión. A partir de este, la grabadora comenzará a registrar toda la navegación y acciones que realicemos sobre el sitio web en cuestión.

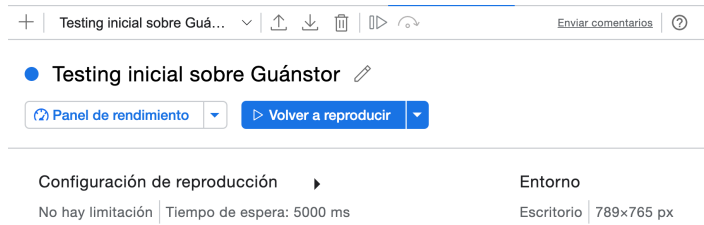
Figura 46. Grabación



Fuente: captura de pantalla de la aplicación DevTools.

Podremos ver el *roadmap* del proceso que realizamos sobre el sitio web en cuestión, para entender el comportamiento de este y generar un *log* o bitácora. Al finalizar con el proceso de registro de *testing* en la aplicación, detenemos la grabación pulsando el botón correspondiente: **Finalizar grabación**.

Figura 47. Testing

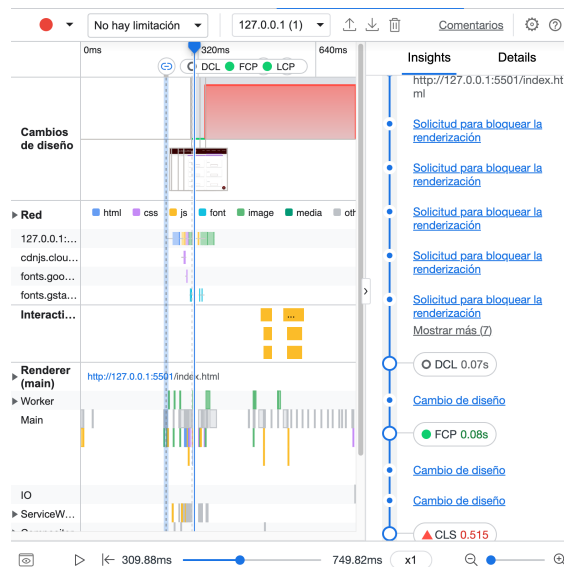


Fuente: captura de pantalla de la aplicación DevTools.

Reproducir un caso de pruebas

Finalmente, podremos acceder a reproducir el caso de pruebas grabado previamente. Su reproducción puede realizarse en velocidad normal, lenta o incluso, muy lenta. Y a través de la misma herramienta, podremos ver un panel de rendimiento para ver cómo responde la interacción de la *webapp* y, entre otros datos, cómo responden los recursos internos y externos que esta *webapp* utiliza, en cuanto a tiempos de carga y uso de la memoria del navegador web.

Figura 48. Reproducción de un caso de pruebas



Fuente: captura de pantalla de la aplicación DevTools.

Todo esto se puede visualizar en un panel de línea de tiempo para ver y analizar en detalle este caso de prueba que hemos realizado.

Importar y exportar los casos de prueba grabados

La misma herramienta nos permite exportar e importar casos de prueba, en formato JSON, para poder enviar evaluaciones o procesos de *testing* a otros compañeros de trabajo o equipos dedicados a estos procesos.

Actividades de repaso

¿Cuál es la principal característica de la herramienta Grabadora?

Facilita la interacción con el sitio web utilizando archivos JSON.

Facilita la grabación de videos en formatos convencionales de la webapp.

Facilita la grabación y reproducción de interacciones de usuario en una webapp.

¿Dónde ubicamos la Grabadora dentro de los navegadores web que la soportan?

En el apartado DevTools > Grabadora.

En el menú Herramientas > Grabadora.

Como una extensión llamada Grabadora.

La herramienta Grabadora permite no solo grabar interacciones, también permite:

Grabar las interacciones exportando directamente a un archivo JSON su resultado.

Reproducir las interacciones, exportarlas e importar interacciones de terceros.

Solo permite grabar y reproducir interacciones, sin posibilidad de almacenarlas.

Video de habilidades

Glosario

Referencias

[Imagen sin título sobre *black friday*]. (s. f.). https://www.freepik.es/vector-premium/disenio-logotipo-compras-linea-black-friday_10512866.htm

[Imagen sin título sobre comando CLI]. (s. f.). <https://blog.expo.dev/new-versioned-expo-cli-cf6e10632656>

[Imagen sin título sobre estadísticas de uso de la web]. (s. f.). <https://www.ltb.media/seo-in-poland/>

[Imagen sin título sobre interfaz de configuración de Expo Client]. (s. f.). <https://bit.ly/3r1SKf>

[Imagen sin título sobre monitoreo de test automatizados]. (s. f.). <https://todosqa.com/tipos-de-automatizacion-de-prueba/>

[Imagen sin título sobre Unit Testing]. (s. f.).

<https://www.shutterstock.com/es/search/functional-software-testing>
